

Bad Genius: Counterfactual-Guided Harness Evolution Beyond Task-Specific Shortcuts

Guojun Zhu^{1,2} Xunheng Huang² Peng Yin³ Jiahui Xie² Sanguo Zhang¹ Doudou Zhou^{2,†}

¹School of Mathematical Sciences, University of Chinese Academy of Sciences, China

²Department of Statistics & Data Science, National University of Singapore, Singapore

³Institute of Automation, Chinese Academy of Sciences, China

[†]Corresponding author: ddzhou@nus.edu.sg

Abstract

Reliable agent evaluation is complicated by automatic harness optimization, which repeatedly uses a released benchmark B_{rel} to guide a Proposer that edits prompts, memory, retrieval, tools, and control code around a fixed target agent. Task holdout varies semantic tasks but leaves the benchmark protocol fixed, so a “bad genius” Proposer can produce a cheating harness whose released-benchmark gain depends on a benchmark-wide shortcut. We introduce **Counterfactual Harness Search and Evolution (CHASE)**, which casts harness evolution as constraint generation over validity-preserving benchmark counterfactuals. After each Proposer update, a *Challenger* searches for an executable protocol transformation with large gain destruction. A validity firewall checks that task semantics are preserved, while a confirmation set determines whether the counterfactual enters a finite archive. We formalize an exact shortcut-neutralized benchmark B_0 and establish statistical guarantees linking finite counterfactual archives to B_0 and characterizing sequential Challenger search. We evaluate CHASE on a synthetic benchmark and on OfficeQA, where CHASE retains strong released-benchmark gains while substantially reducing gain destruction under valid protocol changes.

1 Introduction

Reliable agent evaluation is central to progress in agentic AI, with benchmarks playing a key role in determining which systems appear capable, safe, and ready for deployment. A reported score, however, does not reflect the target agent alone, but also depends on the harness through which it operates and the benchmark protocol under which it is evaluated. A harness specifies the executable context around the agent, including what information is stored and retrieved, how tools and workspace are exposed, and how outputs are handled. Evaluation is increasingly concerned not only with performance under a fixed harness, but also with the performance attainable after optimizing the harness around a fixed target agent. Meta-Harness demonstrates that an agentic Proposer can search directly over this code using harness source code, scores, and execution traces (Lee et al., 2026). Subsequent work studies held-out evaluation, optimizer quality, priority ranking, and reliable harness selection (Wang et al., 2026b; Ong et al., 2026; Ursekar et al., 2026; Zhao et al., 2026). Harness-Bench further shows that performance and failure modes vary materially across agent–harness configurations under shared tasks (Yao et al., 2026). Together, these results make harness optimization—and, more broadly, harness evolution—a substantive evaluation target (Zhu et al., 2026a,b).

Repeated use of benchmark feedback creates a distinct generalization problem. The “Bad genius”¹ Proposer is optimized for benchmark score, not for preserving what that score is intended to measure, and can modify the harness using benchmark feedback and execution traces. Classical shortcut learning concerns predictive rules induced by data or task artifacts (Geirhos et al., 2020; Ribeiro et al., 2020). Harness evolution adds an executable route: the Proposer can write the shortcut into the agent’s protocol. Existing methods ban task identifiers, filenames, and per-task repair recipes, then evaluate on held-out tasks (Wang et al., 2026b; Zhang et al., 2026). These methods limit a *task-specific shortcut*—for example, detecting one question ID and opening a known document—but not a *benchmark-wide shortcut*, such as searching one document channel first because the released protocol places supporting evidence there more

¹The title refers to the film *Bad Genius*, where a gifted student helps others cheat through clever schemes.

often. As illustrated in Figure 1(a), task holdout therefore leaves benchmark-wide shortcuts intact: tasks change, but the protocol correlation does not. We instantiate this problem empirically in document-grounded agent evaluation, where protocol transformations can be executed and validity-checked while task content and scoring remain fixed. For example, 58.1% of questions in the benchmark OfficeQA Full mention numerical scales such as millions or billions. In its 697-document corpus, the next nonblank line after 95.2% of unit statements begins a table (Supplementary C.1). The Proposer may therefore propose a harness that induces the agent to inspect only the text immediately preceding a table for its unit.

Task holdout alone does not evaluate whether the observed gain survives once this shared protocol correlation is broken. The natural next step is therefore to retain held-out tasks while varying the protocol as well. Figure 1(b)–(c) illustrates this comparison. Under the released protocol in Figure 1(b), the shortcut “search channel A first” still places the supporting evidence first. Figure 1(c) then reassigns the same documents across channels, removing this shortcut advantage while keeping the question, answer, and documents fixed. The relevant quantity is not the resulting score change alone, since the benchmark intervention may affect both the evolved and initial harnesses, but how much of the evolved harness’s gain over H_0 disappears. A large loss of gain reveals benchmark dependence that held-out tasks alone cannot expose. This raises a question: *can we construct a benchmark that neutralizes all benchmark-wide shortcuts while preserving the underlying tasks?*

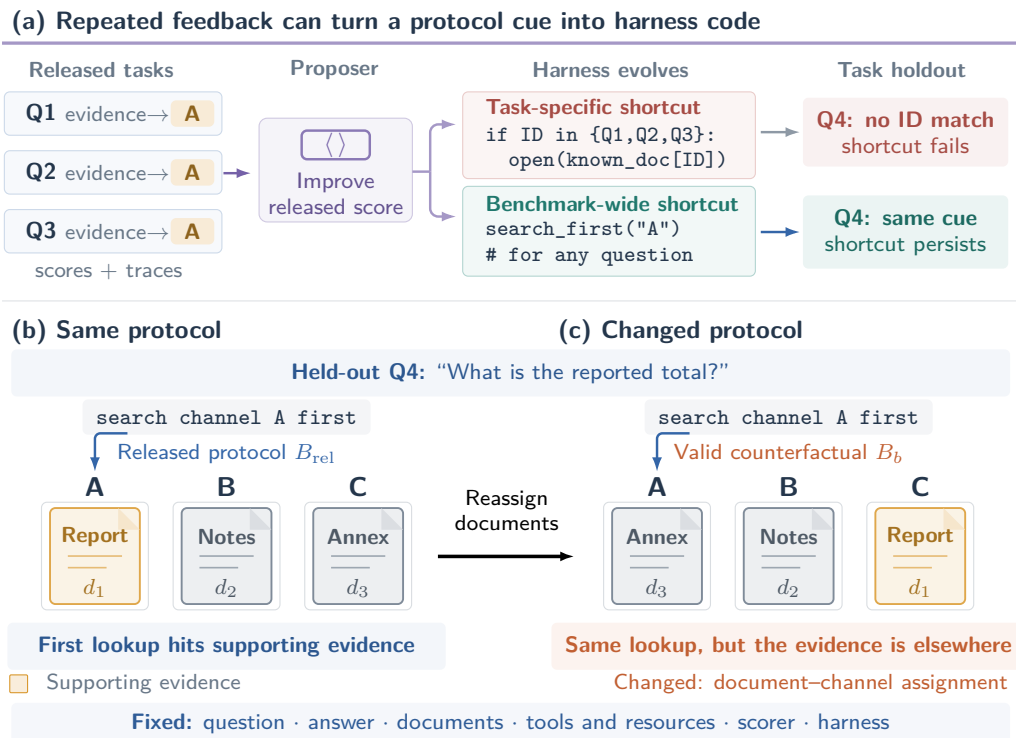


Figure 1: **Motivation: Why task holdout can miss benchmark-wide shortcuts.**

If every shortcut mechanism were known, one could construct an ideal shortcut-neutralized benchmark B_0 and optimize the harness on it. In realistic agent benchmarks, however, benchmark-wide shortcuts can arise from many parts of the protocol and remain hidden from both manual inspection and automated checks, making them difficult to pre-specify with fixed rules. We therefore introduce a separate **Challenger** that searches online, after each harness update, for executable benchmark transformations, each producing a counterfactual benchmark designed to expose shortcuts in the current harness. Whereas prior work introduces auxiliary agents as *debuggers* or *critics* to guide harness repair (Lin et al., 2026; Li et al., 2026), our Challenger attacks shortcut gains at their source—the benchmark protocol. Each valid counterfactual confirmed on held-out tasks becomes a constraint on harness evolution, limiting how much future gains can depend on the same protocol correlation.

Our contributions are fourfold. First, we distinguish task-specific shortcuts from benchmark-wide harness shortcuts and define a gain-destruction estimand that assesses whether observed harness gains remain reliable under benchmark-

preserving counterfactuals. Second, we propose **Counterfactual Harness Search and Evolution (CHASE)**, in which a Challenger searches for executable benchmark counterfactuals. Third, we establish finite-archive guarantees that certify how much harness gain survives shortcut neutralization through counterfactual-guided evolution and when Challenger search can no longer expose a large violation. Fourth, across synthetic benchmark Syn-Ledger and OfficeQA, we show that CHASE yields confirmed counterfactuals and substantially better generalization to protocol variants than other baselines.

2 Related Work

Harness optimization. Earlier work automates LLM-system optimization through metric-driven pipeline compilation in *DSPy* (Khatab et al., 2024), prompt search in *OPRO* (Yang et al., 2024), and feedback propagation in *TextGrad* (Yuksekgonul et al., 2025). *Meta-Harness* optimizes prompts, memory, tools, and control code using evaluation feedback (Lee et al., 2026). *HarnessOpt-Bench* and priority ranking evaluate the optimizer itself under bounded target-agent calls (Ursekar et al., 2026; Ong et al., 2026). *Harness-Bench* isolates configuration-level harness effects across model backends (Yao et al., 2026). *HarnessLens* reduces verification cost by selecting behavior-relevant tasks for each candidate and confirming promising edits on an additional task batch (Xu et al., 2026). AutoSaddler further combines failure-trace diagnosis with validation-based harness updates (Park et al., 2026). These works characterize the optimization object, optimizer quality, and verification efficiency. CHASE addresses a different question: whether feedback from B_{rel} selects a harness whose gain depends on protocol-level correlations shared across tasks.

Harness generalization. *Held-out-task evaluation* shows that optimizing and evaluating harnesses on the same tasks can substantially overstate their gains (Wang et al., 2026b; Esakkiraja et al., 2026). *HarnessCompass* uses a fixed Generalization Gate to allow only task-agnostic modifications, together with component-wise feedback to guide harness updates (Zhang et al., 2026). *HarnessEvolve* uses reference trajectories, quality and performance gates, and held-out validation to filter and select harness updates (Jiang et al., 2026). *EvoSafeHarness* further uses fresh-context adversarial review to reject benchmark-specific safety rules during harness optimization (Li et al., 2026). *Harness continual learning* evaluates new harness updates for retention of previously acquired behavior (Kang et al., 2026). More broadly, continually updated benchmarks, lifelong test sets, and overfitting alarms reduce repeated reliance on a fixed evaluation set (Prabhu et al., 2024; Ishida et al., 2026). Overall, existing methods mainly vary tasks or test content, apply fixed filters to harness edits, or validate candidate updates before adoption. The former leaves the benchmark protocol unchanged, while the latter can only cover shortcut patterns anticipated by the filter. CHASE instead searches for counterfactuals against the current harness while preserving task semantics, without relying on a fixed filter over harness edits.

Benchmark validity. Agent scores are properties of an agent–harness–environment–protocol stack rather than the target agent alone (Zhu et al., 2026a,b; Yao et al., 2026). Recent benchmark audits show that agent scores can be inflated by benchmark-wide shortcuts, search-time contamination, broken tasks, and scoring errors (Shao et al., 2026; Wang et al., 2026c; Dong et al., 2026). Contamination tests and refreshed benchmarks address exposure of benchmark content (Oren et al., 2023; White et al., 2025; Wu et al., 2025). *Auditing Harness Tampering* studies a related problem in self-improving agents, where harness edits can produce apparent performance gains without genuine capability improvement, and develops audits to detect and locate such edits (Wang et al., 2026a). *HackProbe* similarly detects reward hacking during self-evolution and uses the resulting signal for candidate reselection (Yang et al., 2026). These works diagnose or reduce specific sources of invalid benchmark gains. CHASE instead tests protocol dependence directly by searching for valid protocol changes that destroy the gain of the current harness and turning confirmed counterfactuals into constraints on subsequent evolution.

Counterfactual-guided optimization. Counterfactual-guided model repair repeatedly finds a counterfactual to the current model and updates the model to remove it (Bauer-Marquart et al., 2021; Boetius et al., 2023). *Model-written evaluations* and *automated red teaming* use one model to generate tests that expose failures of another (Perez et al., 2022, 2023). *Metamorphic testing* evaluates a system under transformations that keep the task semantics unchanged (Hyun et al., 2023; Cho et al., 2025). CHASE combines these ideas for harness evolution: after each Proposer update, the Challenger searches for a valid benchmark counterfactual that destroys the current harness’s gain, and a confirmed counterfactual constrains subsequent harness updates.

3 Counterfactual Harness Search and Evolution

3.1 Problem Setup

Fix a target agent A , and recall from Section 1 that B_{rel} denotes the released benchmark used for harness optimization. To describe its evaluation target, let U denote the semantic task state and let $Y = \psi(U)$ be the intended target, where $\psi(\cdot)$ is the target rule. The released protocol configuration V_{rel} contains file names, directory layout, metadata, tool aliases, demonstration order, feedback format, and other benchmark-specific details, while Q_{rel} specifies how the agent interacts with these components. Thus, B_{rel} combines a distribution P over semantic tasks U and their induced targets $Y = \psi(U)$, the released protocol $(V_{\text{rel}}, Q_{\text{rel}})$, the available tools and resource budgets, and a scorer. Write $B_{\text{rel}} = (P, V_{\text{rel}}, Q_{\text{rel}}, \psi)$. More generally, for any benchmark variant B sharing the semantic-task distribution P and any harness H , let $\tau^{H,B} \sim p_A(\cdot | H, B, U)$ denote the resulting execution trajectory and let $r_B(\tau, U) \in [0, 1]$ denote the score assigned by benchmark B .

Suppose harness optimization runs for T rounds. Let H_0 be the initial harness. At each round $t = 1, \dots, T$, a Proposer—for example, GPT-5.6 Sol configured as a coding agent—observes the current harness, previous scores, and execution traces, edits H_{t-1} , and returns H_t . Let $D_s = \{U_i\}_{i=1}^{n_s}$ denote the finite search-task set. If $\tau_i^{H_t, B}$ is the rollout of (A, H_t) on U_i , define the observed score $\hat{R}_{B, D_s}(A, H_t)$ and the population score $R_B(A, H)$:

$$\hat{R}_{B, D_s}(A, H_t) := \frac{1}{n_s} \sum_{i=1}^{n_s} r_B(\tau_i^{H_t, B}, U_i), \quad R_B(A, H) := \mathbb{E}_{U \sim P} \mathbb{E}_{\tau^{H, B} \sim p_A(\cdot | H, B, U)} [r_B(\tau^{H, B}, U)].$$

The outer loop therefore evaluates $H_0, H_1, \dots, H_T$ using the scores. After breaking ties by a fixed rule, the search-optimal harness among the evaluated candidates is:

$$H_{\text{rel}}^* \in \left\{ H_t : \hat{R}_{B_{\text{rel}}, D_s}(A, H_t) = \max_{0 \leq j \leq T} \hat{R}_{B_{\text{rel}}, D_s}(A, H_j) \right\}. \quad (1)$$

The Proposer’s direct objective is to increase $\hat{R}_{B_{\text{rel}}, D_s}(A, H_t)$ by modifying the harness, rather than to ensure that each task is solved through the intended capability. Because it can inspect prompts, memory, retrieval logic, code, tool calls, and execution traces, the resulting harness may achieve a higher search score by exploiting a benchmark-wide shortcut.

If a shortcut mechanism is known, an exact transformation may construct a neutralized benchmark $B_0 := \Phi_0(B_{\text{rel}})$ through $\Phi_0(B_{\text{rel}}) = (P, V_0, Q_0, \psi)$. A valid Φ_0 removes the specified protocol-level correlation while leaving U , $Y = \psi(U)$, and the semantic-task distribution P unchanged. Write $R_0 := R_{B_0}$ and $R_{\text{rel}} := R_{B_{\text{rel}}}$. Define $G_{\text{rel}}(H; H_0) = R_{\text{rel}}(A, H) - R_{\text{rel}}(A, H_0)$ and $G_0(H; H_0) = R_0(A, H) - R_0(A, H_0)$. Then, we define $\Delta_{\text{BS}}(H; H_0)$ with:

$$G_{\text{rel}}(H; H_0) = G_0(H; H_0) + \Delta_{\text{BS}}(H; H_0),$$

where $G_{\text{rel}}(H; H_0)$ is the improvement observed during harness optimization, while $G_0(H; H_0)$ is the improvement that survives shortcut neutralization. The remaining $\Delta_{\text{BS}}(H; H_0)$ is the extra gain obtained by the “bad genius” Proposer through increased reliance on the benchmark-wide shortcut rather than improved task-solving capability.

Task holdout probes a different axis (Wang et al., 2026b). Let D_h be a test-task set disjoint from D_s , and let $R_{B, D}(A, H) := \mathbb{E}_{U \sim \hat{P}_D} \mathbb{E}_{\tau \sim p_A(\cdot | H, B, U)} [r_B(\tau, U)]$ denote expected average score on a finite task set D , where P is replaced by the empirical distribution $\hat{P}_D$. The excess search-set gain is:

$$\Delta_{\text{TS}}(H; H_0) = \{R_{B_{\text{rel}}, D_s}(A, H) - R_{B_{\text{rel}}, D_s}(A, H_0)\} - \{R_{B_{\text{rel}}, D_h}(A, H) - R_{B_{\text{rel}}, D_h}(A, H_0)\}.$$

A task-specific shortcut can make $\Delta_{\text{TS}}(H; H_0) > 0$. A benchmark-wide shortcut can persist in both D_s and D_h because both use the same $(V_{\text{rel}}, Q_{\text{rel}})$. Therefore $\Delta_{\text{TS}}(H; H_0) \approx 0$ does not imply $\Delta_{\text{BS}}(H; H_0) \approx 0$. Our goal is to achieve a substantial positive $G_0(H; H_0)$ while keeping $\Delta_{\text{BS}}(H; H_0)$ below a small tolerance.

3.2 From Counterfactuals to Harness Evolution

The exact B_0 above defines the target estimand when the shortcut mechanism and its neutralization are known. In general, neither is available. CHASE therefore replaces the single B_0 with a family of validity-preserving counterfactual

benchmarks. Index each protocol transformation by b . Each Φ_b acts on benchmark configurations; its action on the released benchmark preserves the same semantic task and maps:

$$\Phi_b(B_{\text{rel}}) = (P, V_b, Q_b, \psi), \quad B_b := \Phi_b(B_{\text{rel}}).$$

Write $\text{Valid}(\Phi_b) = 1$ when the transformation preserves the semantic task, target, evidence, resources, and scoring semantics while changing only the declared protocol surface. The valid family is $\mathcal{B}_{\text{val}} = \{B_b = \Phi_b(B_{\text{rel}}) : \text{Valid}(\Phi_b) = 1\}$, and the identity transformation includes B_{rel} in $\mathcal{B}_{\text{val}}$.

Validity Firewall. The benchmark-specific firewall sets $\text{Valid}(\Phi_b) = 1$ only when all executable checks comparing B_{rel} with $\Phi_b(B_{\text{rel}})$ pass; otherwise, the proposal is rejected. These checks include preserving the semantic task and correct answer and restricting changes to the declared protocol surface. The complete set of checks is given in Supplementary B.3.

For any candidate H and $B_b \in \mathcal{B}_{\text{val}}$, let $R_b = R_{B_b}$ and define $G_b(H; H_0) = R_b(A, H) - R_b(A, H_0)$. Taking $B_b = B_{\text{rel}}$ or $B_b = B_0$ gives $G_{\text{rel}}(H; H_0)$ or $G_0(H; H_0)$, respectively. The released gain destroyed by B_b is $\Delta_b(H; H_0) = G_{\text{rel}}(H; H_0) - G_b(H; H_0)$, which is the central estimand. When $B_b = B_0$, the contrast reduces to $\Delta_{\text{BS}}(H; H_0)$, so the counterfactual family directly extends exact neutralization. Negative Δ_b means that the harness gain increases rather than decreases under the counterfactual.

CHASE then alternates between a Proposer and a Challenger. Since $\mathcal{B}_{\text{val}}$ is an unknown infinite family, before round t , the Proposer has access to a finite archive $\mathcal{A}_{t-1} \subset \mathcal{B}_{\text{val}}$ containing B_{rel} and all previously confirmed counterfactuals. At the population level, the ideal Proposer solves:

$$H_t \in \arg \max_H G_{\text{rel}}(H; H_0) \quad \text{s.t.} \quad \Delta_b(H; H_0) \leq \varepsilon, \quad \forall B_b \in \mathcal{A}_{t-1}, \quad (2)$$

where $\varepsilon \geq 0$ controls how much of the released-benchmark gain may disappear under any archived counterfactuals. The ideal Challenger targets the resulting H_t by searching for an executable violation,

$$B_{b_t} \in \arg \max_{B_b \in \mathcal{B}_{\text{val}}} \Delta_b(H_t; H_0), \quad (3)$$

and returns code for transformation Φ_{b_t} . The criterion in equation 1 is inadequate for CHASE, since a high-scoring harness may violate counterfactual constraints discovered in later rounds. After updating the archive, we therefore select the final harness from those evaluated during evolution:

$$H^* \in \arg \max_{H \in \{H_0, H_1, \dots, H_T\}} G_{\text{rel}}(H; H_0) \quad \text{s.t.} \quad \Delta_b(H; H_0) \leq \varepsilon, \quad \forall B_b \in \mathcal{A}_T. \quad (4)$$

Before final selection, we evaluate any missing harness–benchmark pairs among $H_0, \dots, H_T$ and $\mathcal{A}_T$ on D_{evo} , and then apply the archive-feasibility rule.

3.3 Finite Archive and Statistical Confirmation

The objectives in equations 2–4 above are written at the population level; implementation and the analysis below use their empirical counterparts. For any finite task set D , define $\hat{R}_{B,D}(A, H) := \frac{1}{|D|} \sum_{U_i \in D} r_B(\tau_i^{H,B}, U_i)$, $\hat{G}_{b,D}(H; H_0) := \hat{R}_{B_b,D}(A, H) - \hat{R}_{B_b,D}(A, H_0)$, and $\hat{\Delta}_{b,D}(H; H_0) := \hat{G}_{\text{rel},D}(H; H_0) - \hat{G}_{b,D}(H; H_0)$. When a task is evaluated with multiple rollouts, the rollout scores are first averaged within task. To separate search, confirmation, and final certification, the evaluation tasks are assigned disjoint roles:

$$D_{\text{evo}}, D_{\text{disc}}, D_{\text{conf},1}, \dots, D_{\text{conf},T}, D_{\text{cert}} \quad \text{are mutually disjoint.}$$

Here D_{evo} supplies the feedback used by the Proposer to search H_t in equation 2, D_{disc} supplies the feedback used by the Challenger to search for Φ_{b_t} in equation 3, $D_{\text{conf},t}$ is used to confirm Φ_{b_t} after it has been fixed, and D_{cert} remains sealed until final certification. This separation reserves fresh tasks for confirmation and final certification, extending the search–test distinction (D_s, D_h) in Section 3.1.

Since $\mathcal{B}_{\text{val}}$ is unknown and cannot be exhaustively searched, CHASE maintains the sequence of finite archives $\{\mathcal{A}_t\}_{t=0}^T$. Initialize $\mathcal{A}_0 = \{B_{\text{rel}}\}$. Once the Challenger proposes Φ_{b_t} , its executable code is fixed and its contrast is

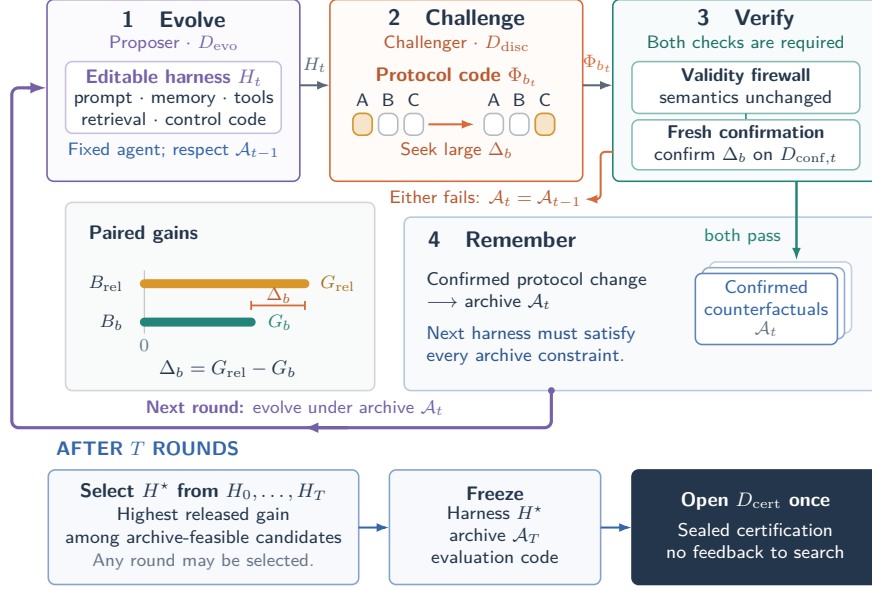


Figure 2: An overview of CHASE.

re-estimated on $D_{\text{conf},t}$. For a confirmation threshold $\eta_{\text{conf},t} > 0$, let $\text{Conf}_t := \mathbf{1}\{\hat{\Delta}_{b_t, D_{\text{conf},t}}(H_t; H_0) \geq \eta_{\text{conf},t}\}$. The archive is then updated by:

$$\mathcal{A}_t = \begin{cases} \mathcal{A}_{t-1} \cup \{B_{b_t}\}, & \text{Valid}(\Phi_{b_t}) = 1 \text{ and } \text{Conf}_t = 1, \\ \mathcal{A}_{t-1}, & \text{otherwise.} \end{cases} \quad (5)$$

In practice, each archived B_{b_t} is stored together with the executable code for Φ_{b_t} . Thus a Challenger proposal becomes a constraint on subsequent harness evolution only when it is valid and its effect is confirmed on $D_{\text{conf},t}$.

To summarize performance over any finite counterfactual archive $\mathcal{A} \subset \mathcal{B}_{\text{val}}$, define:

$$\Gamma_{\mathcal{A}}(H; H_0) := \max_{B_b \in \mathcal{A}} \Delta_b(H; H_0), \quad G_{\min, \mathcal{A}}(H; H_0) := \min_{B_b \in \mathcal{A}} G_b(H; H_0),$$

with empirical counterparts on a task set D ,

$$\hat{\Gamma}_{\mathcal{A}, D}(H; H_0) := \max_{B_b \in \mathcal{A}} \hat{\Delta}_{b, D}(H; H_0), \quad \hat{G}_{\min, \mathcal{A}, D}(H; H_0) := \min_{B_b \in \mathcal{A}} \hat{G}_{b, D}(H; H_0).$$

Here, $\Gamma_{\mathcal{A}}(H; H_0)$ is the largest gain destruction, $G_{\min, \mathcal{A}}(H; H_0)$ is the smallest surviving gain over $\mathcal{A}$, and we have $G_{\min, \mathcal{A}}(H; H_0) + \Gamma_{\mathcal{A}}(H; H_0) = G_{\text{rel}}(H; H_0)$. The set D_{cert} is used once to evaluate the constraints in $\mathcal{A}_T$. Figure 2 summarizes the complete CHASE information flow.

4 Statistical Guarantees

Theoretical analyses of harness self-evolution remain limited. Recent work studies finite-data certification and safe adoption under a fixed task distribution, focusing on expected-reward improvement and retention of prior behavior (Cai et al., 2026). It does not model benchmark-protocol dependence or counterfactual discovery. We therefore ask what a finite counterfactual archive can certify about B_0 and, when it does not yet recover B_0 , what can be concluded from sequential Challenger search.

For valid $\Phi_b, \Phi_{b'}$, define $B_{b'ob} := \Phi_{b'}(\Phi_b(B_{\text{rel}}))$ by applying Φ_b first and $\Phi_{b'}$ second. This composition maps $(P, V_{\text{rel}}, Q_{\text{rel}}, \psi)$ to $(P, V_{b'ob}, Q_{b'ob}, \psi)$, and we assume $\mathcal{B}_{\text{val}}$ is closed under it. For nonempty finite $\mathcal{A} \subset \mathcal{B}_{\text{val}}$, define:

$$K(\mathcal{A}) := \min \{k \in \mathbb{N}^+ : \exists B_{b_1}, \dots, B_{b_k} \in \mathcal{A}, B_{b_k \circ \dots \circ b_1} = B_0\}.$$

Thus $K(\mathcal{A}) = \infty$ if no composition of transformations in $\mathcal{A}$ produces B_0 . Also define:

$$\rho := \sup_H \sup_{B_b, B_{b'} \in \mathcal{B}_{\text{val}}} [\Delta_{b' \circ b}(H; H_0) - \Delta_b(H; H_0) - \Delta_{b'}(H; H_0)]_+.$$

Thus $K(\mathcal{A})$ is the smallest number of archived transformations whose composition produces B_0 , while ρ measures the worst-case excess gain destruction under composition. Equivalently, $\Delta_{b' \circ b}(H; H_0) \leq \Delta_b(H; H_0) + \Delta_{b'}(H; H_0) + \rho$. Fix $\alpha \in (0, 1)$. We obtain the following results.

Theorem 1 *Let $\mathcal{A} \subset \mathcal{B}_{\text{val}}$ be nonempty and finite. If $\mathcal{A}$ and H^* are fixed before D_{cert} is opened and $K(\mathcal{A}) < \infty$, then, under the conditions in Supplementary A, with probability at least $1 - \alpha/2$:*

$$\begin{aligned} \Delta_{\text{BS}}(H^*; H_0) &\leq K(\mathcal{A}) \left\{ \widehat{\Gamma}_{\mathcal{A}, D_{\text{cert}}}(H^*; H_0) + \sqrt{8 \log(4|\mathcal{A}|/\alpha)/|D_{\text{cert}}|} \right\} + (K(\mathcal{A}) - 1)\rho, \\ G_0(H^*; H_0) &\geq \widehat{G}_{\min, \mathcal{A}, D_{\text{cert}}}(H^*; H_0) - \sqrt{2 \log(4|\mathcal{A}|/\alpha)/|D_{\text{cert}}|} \\ &\quad - (K(\mathcal{A}) - 1) \left\{ \widehat{\Gamma}_{\mathcal{A}, D_{\text{cert}}}(H^*; H_0) + \sqrt{8 \log(4|\mathcal{A}|/\alpha)/|D_{\text{cert}}|} + \rho \right\}. \end{aligned}$$

Theorem 1 converts a finite archive into guarantees for B_0 . The bounds tighten as $K(\mathcal{A})$ and ρ decrease. If $B_0 \in \mathcal{A}$, then $K(\mathcal{A}) = 1$ and every term containing ρ disappears.

For each search set D , let $\mathfrak{R}_D$ denote the expected absolute Rademacher complexity of its released-score and gain-destruction functions, defined in Supplementary A.1. We use $r = \max_{D \in \{D_{\text{evo}}, D_{\text{disc}}\}} \left\{ 2\mathfrak{R}_D + \sqrt{8 \log(8/\alpha)/|D|} \right\}$ which bounds the empirical-to-population deviation of the released score and gain destruction over the candidates considered. For confirmation, set $\eta_{\text{conf}, t} = \varepsilon + \gamma + \sqrt{8 \log(8T/\alpha)/|D_{\text{conf}, t}|}$, $0 < \gamma \leq 1$. Supplementary A discusses $\mathfrak{R}_D$ and γ .

Theorem 2 *Suppose $\text{Valid}(\Phi_{b_t}) = 1$ and $\widehat{\Delta}_{b, D_{\text{evo}}}(H_t; H_0) \leq \varepsilon$ for every $B_b \in A_{t-1}$ at each round. Assume that, for every $0 < q \leq 1$, any m harnesses with $\sup_{B_b \in \mathcal{B}_{\text{val}}} |\Delta_b(H^{(j)}; H_0) - \Delta_b(H^{(k)}; H_0)| \geq q$, for all $j \neq k$, obey $m \leq (C/q)^d$, for constants $C \geq 1$ and $d > 0$. If $r < \gamma$, then, with probability at least $1 - \alpha/2$,*

$$\#\{t \leq T : \text{Conf}_t = 1\} \leq \left(\frac{C}{\gamma - r} \right)^d.$$

Consequently, if $T > \left(\frac{C}{\gamma - r} \right)^d$, then $\text{Conf}_t = 0$ for at least one $t \leq T$.

Theorem 2 shows that each counterfactual forces a nontrivial change in the subsequent harness, so such admissions cannot continue indefinitely. In particular, for a sufficiently large T , the process must reach a round with $\text{Conf}_t = 0$. We next characterize what can be concluded at such a round.

Theorem 3 *Suppose H_t and B_{b_t} solve the empirical counterparts of equation 2 and equation 3. On the common evolution event, of probability at least $1 - \alpha/2$, every round with a valid tested proposal and $\text{Conf}_t = 0$ satisfies:*

$$\begin{aligned} \sup_{B_b \in \mathcal{B}_{\text{val}}} \Delta_b(H_t; H_0) &< \varepsilon + \gamma + 2r + 2\sqrt{8 \log(8T/\alpha)/|D_{\text{conf}, t}|}, \\ G_{\text{rel}}(H_t; H_0) &\geq \sup_{\tilde{H}: \Delta_b(\tilde{H}; H_0) \leq \varepsilon - r, \forall B_b \in \mathcal{B}_{\text{val}}} G_{\text{rel}}(\tilde{H}; H_0) - 2r. \end{aligned}$$

Under these optimization conditions, Theorem 3 gives the complementary conclusion: a non-confirmed round certifies small gain destruction over the full valid family, up to estimation error, while retaining near-optimal released gain among uniformly feasible harnesses. Thus, confirmed rounds expand the archive, whereas a non-confirmed round provides an approximate stopping certificate for the current harness.

5 Experiments

5.1 Benchmarks and experimental setup

OfficeQA is a new benchmark suite for end-to-end grounded reasoning over dense U.S. government financial documents (Opsahl-Ong et al., 2026). Its questions require document discovery, text and table retrieval, numerical reasoning,

and exact answer extraction. OfficeQA Full has 246 questions; Pro V2 adds 90 questions over a separate receipts-and-expenditures corpus (Databricks, 2026). Full supports harness optimization, and Pro V2 provides a challenging public evaluation.

OfficeQA is sensitive to harness design because search, evidence handling, tool use, computation, and answer formatting are harness-controlled. At a fixed model, EnvHarness improved OfficeQA exact match from 54.40 to 56.20 and token-level F1 from 55.77 to 57.73, compared with skills extracted from the original environments (Huang et al., 2026). We retain all released questions and answers and let the agent A search the full 697-document transformed-text corpus for OfficeQA Full. Supplementary C gives release, corpus, and scoring details. The 246 questions are divided into evolution, Challenger discovery, Challenger confirmation, and final certification sets of sizes 49, 49, 72, and 76, respectively. We use a fixed budget of three Proposer rounds for all optimized methods. The confirmation set is partitioned into three disjoint round-specific blocks, each of size 24. Pro V2 is reported separately as a cross-corpus analysis. Our use of task splits follows prior OfficeQA evaluations (Alzubi et al., 2026; Ursekar et al., 2026); we additionally report Pro V2 results, which were not reported in those studies.

To isolate protocol shortcuts in this type of multi-document numerical reasoning, we also construct Syn-Ledger with 320 synthetic ledger tasks. Controlled protocol cues allow us to construct B_0 and directly measure how much harness gain survives their neutralization (Supplementary D).

5.2 Methods and evaluation

On OfficeQA, we compare three methods. **RawHarness** keeps the initial harness H_0 unchanged and provides the reference for all gain and gain-destruction metrics. **HarnessCompass** optimizes H_0 for released-benchmark performance under the fixed Generalization Gate of HarnessCompass (Zhang et al., 2026). **CHASE** uses the same Proposer backbone without the fixed Gate; an online Challenger supplies confirmed counterfactuals that enter $\mathcal{A}_t$ and constrain subsequent Proposer rounds and final harness selection. Syn-Ledger includes these three methods and adds B_0 -**Access** with access to B_0 . It retains the same finite search budget and fixed Generalization Gate and is not an upper bound. Within each benchmark, all methods share the target agent A and initial harness H_0 , and optimized methods use the same three-round Proposer backbone. Section 4 provides guidance on threshold calibration when evaluation budgets are sufficiently large. Under our limited budget, we fix $\eta_{\text{conf},t} = 0.075$ and $\varepsilon = 0.05$ on OfficeQA. We report other details in Supplementary B.

On OfficeQA, the primary evaluation applies each method’s final harness H to the 76 certification questions under every benchmark in the final CHASE archive $\mathcal{A}_3$. We report the released-protocol score $\hat{R}_{B_{\text{rel}}, D_{\text{cert}}}(\mathbf{A}, H)$, the average and worst-case scores over $\mathcal{A}_3$. For a finite archive $\mathcal{A}$, define $\hat{R}_{\text{avg}, \mathcal{A}, D_{\text{cert}}}(\mathbf{A}, H) := |\mathcal{A}|^{-1} \sum_{B_b \in \mathcal{A}} \hat{R}_{B_b, D_{\text{cert}}}(\mathbf{A}, H)$ and $\hat{R}_{\text{min}, \mathcal{A}, D_{\text{cert}}}(\mathbf{A}, H) := \min_{B_b \in \mathcal{A}} \hat{R}_{B_b, D_{\text{cert}}}(\mathbf{A}, H)$. For brevity, we write the three certification metrics as $\hat{R}_{\text{rel}}$, $\hat{R}_{\text{avg}, \mathcal{A}_3}$ and $\hat{R}_{\text{min}, \mathcal{A}_3}$. Separately, $\hat{R}_{\text{ProV2}}$ denotes the released-protocol scores on the 90-question Pro V2 release. Details are given in Supplementary C. We also compare token use and wall time for both the search stage and final certification, with further details in Supplementary C.5.

On Syn-Ledger, we evaluate all four methods on the 208 certification tasks under B_{rel} and B_0 . We report the $\hat{R}_{\text{rel}}$, $\hat{R}_0$, $\hat{G}_{\text{rel}}$, $\hat{G}_0$, and $\hat{\Delta}_{\text{BS}}$, with certification-set and harness arguments suppressed. Supplementary D gives the task-allocation details. For metrics $\hat{G}_{\text{rel}}$, $\hat{G}_0$, and $\hat{\Delta}_{\text{BS}}$.

5.3 Results

Table 1 compares performance on the separate Pro V2 releases with certification across the final archive. On D_{cert} , CHASE achieves the highest released score and remains stable across the confirmed counterfactuals. In contrast, HarnessCompass performs below the initial harness, suggesting that its fixed generalization gate does not ensure improved held-out performance on OfficeQA. This is a new evaluation setting for HarnessCompass, whose original experiments focus on SWE-bench Verified (Jimenez et al., 2024; Chowdhury et al., 2024); our implementation of its generalization gate is detailed in Supplementary C.4. The advantage of CHASE carries over to the separate Pro V2 corpus, where it scores 30.37%, compared with 26.30% for HarnessCompass.

The first-round Challenger proposes collecting a table’s associated context before the table and supplies an executable declarative transformation specification. It hypothesizes that the evolved harness H_1 , instructed to “keep an explicit unit for every operand,” may rely on customary locations of units and notes. Figure 3 illustrates the proposal using a real OfficeQA excerpt; the interpreter executes the generated specification by re-encoding retrieved text as table context in the agent-visible JSON search results (Supplementary C.3). On D_{evo} , H_1 ’s gain over H_0 decreases from 8.16% under

Table 1: OfficeQA results. All evaluations use three rollouts per question.

Method	Separate release	Certification of OfficeQA		
	$\widehat{R}_{\text{ProV2}}(\uparrow)$	$\widehat{R}_{\text{rel}}(\uparrow)$	$\widehat{R}_{\text{avg}, \mathcal{A}_3}(\uparrow)$	$\widehat{R}_{\text{min}, \mathcal{A}_3}(\uparrow)$
RawHarness	27.04%	67.98%	66.23%	64.47%
HarnessCompass	26.30%	64.04%	63.16%	62.28%
CHASE	30.37%	68.86%	68.42%	67.98%

B_{rel} to -5.10% under B_{b_1} . This reversal shows that H_1 's released-benchmark gain depends on how retrieved evidence is represented. Once B_{b_1} enters the archive, all second-round proposer candidates violate the $\epsilon = 0.05$ constraint, so CHASE falls back to H_0 . In the third round, one candidate recovers a 4.08% released gain while satisfying the archived constraint and is selected as H_3 .

TABLE ESF-2.—Income and Expense
[In thousands of dollars. Source: Office of the Assistant Secretary of the Treasury for Management]

	Current quarter Jan. 1, 2011, through Mar. 31, 2011
Income and expense	
Profit (+) or loss (-) on:	
Foreign exchange	603,216
Adjustment for change in valuation of SDR holdings and allocations ¹	75,671

¹ Beginning July 1974, the International Monetary Fund adopted a technique for valuing the SDRs based on a weighted average of exchange rates for the currencies of selected member countries. The U.S. SDR holdings and allocations are valued on this basis beginning July 1974.

Note. — Annual balance sheets for fiscal years 1934 through 1940 appeared in the 1940 "Annual Report of the Secretary of the Treasury" and those for succeeding years appeared in subsequent reports through 1980. Quarterly balance sheets beginning with December 31, 1938, have been published in the "Treasury Bulletin." Data from inception to September 30, 1978, may be found on the statements published in the January 1979 "Treasury Bulletin."

Φ_{b_1}

TABLE ESF-2.—Income and Expense
[In thousands of dollars. Source: Office of the Assistant Secretary of the Treasury for Management]

¹ Beginning July 1974, the International Monetary Fund adopted a technique for valuing the SDRs based on a weighted average of exchange rates for the currencies of selected member countries. The U.S. SDR holdings and allocations are valued on this basis beginning July 1974.

Note. — Annual balance sheets for fiscal years 1934 through 1940 appeared in the 1940 "Annual Report of the Secretary of the Treasury" and those for succeeding years appeared in subsequent reports through 1980. Quarterly balance sheets beginning with December 31, 1938, have been published in the "Treasury Bulletin." Data from inception to September 30, 1978, may be found on the statements published in the January 1979 "Treasury Bulletin."

	Current quarter Jan. 1, 2011, through Mar. 31, 2011
Income and expense	
Profit (+) or loss (-) on:	
Foreign exchange	603,216
Adjustment for change in valuation of SDR holdings and allocations ¹	75,671

Figure 3: Illustration of the Challenger's proposal using a real OfficeQA excerpt.

Table 2 shows that B_0 -Access achieves a slightly lower $\widehat{\Delta}_{\text{BS}}$, it directly uses the known B_0 . In contrast, CHASE has no access to B_0 during optimization but still achieves the highest $\widehat{R}_0$. This indicates that its gains transfer well beyond the released benchmark. The negative $\widehat{\Delta}_{\text{BS}}$ reflects the larger gain under B_0 , not an easier benchmark: B_0 yields lower scores than B_{rel} .

Table 2: Syn-Ledger results. All final evaluations use three rollouts per task.

Method	$\widehat{R}_{\text{rel}}(\uparrow)$	$\widehat{R}_0(\uparrow)$	$\widehat{G}_{\text{rel}}(\uparrow)$	$\widehat{G}_0(\uparrow)$	$\widehat{\Delta}_{\text{BS}}(\downarrow)$
RawHarness	81.89%	15.22%	—	—	—
HarnessCompass	80.45%	14.74%	-1.28%	+2.24%	-3.53%
B_0 -Access	83.01%	32.69%	+1.28%	+20.19%	-18.91%
CHASE	87.18%	38.30%	+5.29%	+23.08%	-17.79%

6 Discussion

Harness optimization changes what must generalize. Unlike parameter optimization, where shortcuts typically arise from data or task artifacts (Geirhos et al., 2020), harness evolution searches over executable programs. As a result, harness optimization can introduce benchmark-wide shortcut (Yao et al., 2026; Shao et al., 2026). CHASE makes this distinction explicit: task generalization asks whether a final harness works across U , whereas CHASE asks whether its harness-evolution gain persists under valid $B_b \in \mathcal{B}_{\text{val}}$. This claim is relative to the evaluated counterfactual family, whose coverage is limited by our three-round search budget. Larger budgets may reveal additional protocol dependencies, while stricter validity checks strengthen the credibility of the resulting counterfactuals. Our empirical comparison focuses on HarnessCompass as the closest existing harness-evolution method that explicitly targets generalization beyond the task-specific shortcuts. Broader comparisons with emerging harness optimizers would further characterize how protocol-level counterfactual constraints interact with different evolution strategies.

References

- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. EvoSkill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.
- Fabian Bauer-Marquart, David Boetius, Stefan Leue, and Christian Schilling. Specrepair: Counter-example guided safety repair of deep neural networks. *arXiv preprint arXiv:2106.01917*, 2021.
- David Boetius, Stefan Leue, and Tobias Sutter. A robust optimisation perspective on counterexample-guided repair of neural networks. *arXiv preprint arXiv:2301.11342*, 2023.
- Qianshu Cai, Yonggang Zhang, Jun Nie, Maohao Ran, Huajiang Zheng, Jun Song, Xinmei Tian, Yike Guo, and Wei Xue. Safe harness self-evolution: A theoretical analysis of feasibility and limits. *arXiv preprint arXiv:2609.08175*, 2026.
- Steven Cho, Stefano Ruberto, and Valerio Terragni. Metamorphic testing of large language models for natural language processing. In *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution*, pages 174–186. IEEE, 2025. doi: 10.1109/ICSME64153.2025.00025.
- Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubei, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench verified. OpenAI, August 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- Databricks. OfficeQA: A grounded reasoning benchmark suite. <https://github.com/databricks/officeqa>, 2026. Accessed 2026-09-01.
- Zihan Dong, Zhiyuan Ma, Zekun Wang, Yunqing Li, Zirou Liu, Ruixuan Deng, Qishi Zhan, and Rui Qian. How benchmarks mis-score computer-use agents. *arXiv preprint arXiv:2607.28367*, 2026.
- Esakkivel Esakkiraja, Denis Akhiyarov, Vikas Yadav, Sai Rajeswar, Patrice Bechard, Sridhar Nemala, and Sagar Davasam. StarHarness: Evolving harnesses with stratified search for enterprise environments. *arXiv preprint arXiv:2608.24804*, 2026.
- Robert Geirhos, Jørn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- Chengsong Huang, Zifeng Wang, Rujun Han, Jun Yan, Yanfei Chen, Zoey CuiZhu, Ke Jiang, Peng Xia, Han Yu, Yufan Zhuang, Yifei Ming, Jiaqi Pan, Bhavana Dalvi Mishra, Jiaxin Huang, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. EnvHarness: Awakening static worlds for agent learning. *arXiv preprint arXiv:2608.19880*, 2026.
- Sangwon Hyun, Mingyu Guo, and M. Ali Babar. METAL: Metamorphic testing framework for analyzing large-language model qualities. *arXiv preprint arXiv:2312.06056*, 2023.
- Takashi Ishida, Thanawat Lodkaew, and Ikko Yamane. Capbencher: Give your LLM benchmark a built-in alarm for test-set overfitting. In *International Conference on Machine Learning*, 2026.
- Wen Jiang, Mingmin Chu, Yimeng Tian, Qianxin Zhang, Haofei Yang, Rui Yang, Yang Liu, Tao Lv, and Fangming Li. HarnessEvolve: Learning from reference trajectories for reliable agent self-evolution. *arXiv preprint arXiv:2609.00829*, 2026.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- Borui Kang, Jinrui Gu, Junhan Lv, Wenbin Li, Lei Wang, and Yang Gao. Harness continual learning: Continual adaptation beyond model parameters. *arXiv preprint arXiv:2608.19013*, 2026.

- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=sY5N0zY5Od>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- Nanxi Li, Yingzi Ma, Yulong Cao, Edward Suh, Bo Li, Dawn Song, and Chaowei Xiao. EvoSafeHarness: Evolving model- and domain-specific harnesses for securing agents. *arXiv preprint arXiv:2609.05903*, 2026.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic Harness Engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026. doi: 10.48550/arXiv.2604.25850. URL <https://arxiv.org/abs/2604.25850>.
- Kai Tzu-iunn Ong, Minseok Kang, Dongwook Choi, Junhee Cho, Seungju Kim, Seungwon Lim, Geunha Jang, Minwoo Oh, Bogyung Jeong, Sunghwan Kim, Taeyoon Kwon, and Jinyoung Yeo. Towards direct evaluation of harness optimizers via priority ranking. *arXiv preprint arXiv:2605.22505*, 2026.
- Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. OfficeQA Pro: An enterprise benchmark for end-to-end grounded reasoning. *arXiv preprint arXiv:2603.08655*, 2026.
- Yonatan Oren, Nicole Meister, Niladri Chatterji, Faisal Ladhak, and Tatsunori B. Hashimoto. Proving test set contamination in black box language models. *arXiv preprint arXiv:2310.17623*, 2023.
- Sungho Park, Wonjoong Kim, Rongyuan Tan, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. AutoSaddler: Automatic harness optimization with durable updates from agent execution traces. *arXiv preprint arXiv:2608.23041*, 2026.
- Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. *arXiv preprint arXiv:2202.03286*, 2022.
- Ethan Perez, Sam Ringer, Kamilé Lukošiušė, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, et al. Discovering language model behaviors with model-written evaluations. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13387–13434. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.findings-acl.847.
- Ameya Prabhu, Vishaal Udandara, Philip Torr, Matthias Bethge, Adel Bibi, and Samuel Albanie. Lifelong benchmarks: Efficient model evaluation in an era of rapid progress. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442.
- Jiaqi Shao, Hanck Chen, Wei Zhang, Maxm Pan, and Bing Luo. Do agent benchmarks measure capability? protocol validity in the age of agentic AI. *arXiv preprint arXiv:2607.22368*, 2026.
- Varun Ursekar, Apaar Shanker, Yash Maurya, Shehab Yasser, Vijay S. Kalmath, Veronica Chatrath, and Yuan Xue. Harnessopt-bench: Evaluating LLMs at harness optimization. *arXiv preprint arXiv:2608.06301*, 2026.
- Xing Wang, Xiaoyi Zhang, and Jie Shao. Auditing harness tampering in self-improving agents. *arXiv preprint arXiv:2609.00069*, 2026a.
- Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents. *arXiv preprint arXiv:2607.12227*, 2026b.

- Yongjie Wang, Xinyue Zhang, Kunhong Yao, Zhiwei Zeng, Kaisong Song, Jun Lin, and Zhiqi Shen. Search-time contamination in deep research agents: Measuring performance inflation in public benchmark evaluation. *arXiv preprint arXiv:2606.05241*, 2026c.
- Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddhartha Naidu, et al. Livebench: A challenging, contamination-limited LLM benchmark. In *International Conference on Learning Representations*, 2025.
- Xiaobao Wu, Liangming Pan, Yuxi Xie, Ruiwen Zhou, Shuai Zhao, Yubo Ma, Mingzhe Du, Rui Mao, Anh Tuan Luu, and William Yang Wang. Antileak-bench: Preventing data contamination by automatically constructing benchmarks with updated real-world knowledge. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.
- Jinghan Xu, Yikai Zhang, Aili Chen, Weiyuan Li, Jiaqing Liang, and Deqing Yang. Verify smarter, evolve further: Efficient harness evolution through behavior-aware verification. *arXiv preprint arXiv:2608.27311*, 2026.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Bb4VGOWELI>.
- Rongxin Yang, Yang Liu, Shang Luo, Haoxuan Jia, Chongyang Zhang, Hao Zheng, Yingguang Yang, Yulin Huang, Jianshen Zhang, Yongzhi Qi, Kefu Xu, Congjing Ran, and Bin Chong. Harness-agnostic detection and immunization of reward hacking in self-evolving language models. *arXiv preprint arXiv:2609.04665*, 2026.
- Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Guangxiang Zhao, Lin Sun, Xiangzheng Zhang, and Tong Yang. Harness-bench: Measuring harness effects across models in realistic agent workflows. *arXiv preprint arXiv:2605.27922*, 2026.
- Mert Yuksekogonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639(8055):609–616, 2025. doi: 10.1038/s41586-025-08661-4.
- Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, Yizhou Jin, and Yan Xu. Harnesscompass: Guiding automatic harness evolution toward generalizable and effective agent harnesses. *arXiv preprint arXiv:2608.01918*, 2026.
- Cen Mia Zhao, Haibo Ruan, Wenjie Chen, Pei-fen Tu, Usman Abbasi, and Joel Hesch. Beyond prompts: Measuring and optimizing LLM tool-agent harnesses. *arXiv preprint arXiv:2609.05736*, 2026.
- Pengyu Zhu, Lijun Li, Yaxing Lyu, Qianxin Luo, Jingyi Yang, Yi Liu, Tingfeng Hui, Xinyu Yuan, Li Sun, Sen Su, and Jing Shao. A unified framework for the evaluation of LLM agentic capabilities. *arXiv preprint arXiv:2605.27898*, 2026a.
- Pengyu Zhu, Li Sun, Philip S. Yu, and Sen Su. “LLM Agent Performance” is not a single evaluation target. *arXiv preprint arXiv:2602.03238*, 2026b.

Supplementary Material

Section A contains the proofs, and Section B describes the method implementations. Sections C and D give the OfficeQA experimental details and the Syn-Ledger benchmark construction and evaluation, respectively.

A Theory and Proofs

Section A.1 establishes the common concentration event for empirical evolution. Sections A.2–A.4 prove Theorems 1–3, respectively, covering finite-archive certification, the number of admitted rounds, and guarantees for non-confirmed rounds.

The complexity $\mathfrak{R}_D$ captures the uniform-search error. For example, for a fixed-complexity bounded evaluation class, $\mathfrak{R}_D$ is typically of order $|D|^{-1/2}$ up to complexity factors. Hence, with sufficiently large search sets, r becomes small enough to choose a positive margin $\gamma > r$, which yields the guarantee.

Condition S1 All task sets have positive sizes fixed before they are opened. For each $D \in \{D_{\text{evo}}, D_{\text{disc}}, D_{\text{conf},1}, \dots, D_{\text{conf},T}, D_{\text{cert}}\}$, let $\mathcal{F}_D^-$ denote the information available before D is opened. Conditional on $\mathcal{F}_D^-$, the task-level evaluation vectors are independent across $U_i \in D$, with $U_i \stackrel{\text{iid}}{\sim} P$, and for every admissible evaluation of (H, B) ,

$$0 \leq r_B(\tau_i^{H,B}, U_i) \leq 1, \quad \mathbb{E} \left[r_B(\tau_i^{H,B}, U_i) \mid \mathcal{F}_D^-, U_i \right] = \mathbb{E}_{\tau \sim p_{\Lambda}(\cdot \mid H, B, U_i)} [r_B(\tau, U_i)].$$

The admissible evaluation class for D_{evo} and D_{disc} is fixed before the corresponding data are observed. Moreover, (H_t, b_t) is $\mathcal{F}_{D_{\text{conf},t}}^-$ -measurable, and (H^*, A) is $\mathcal{F}_{D_{\text{cert}}}^-$ -measurable. Dependence among evaluations of the same task is unrestricted.

Condition S1 is only a technical condition for the concentration analysis; it allows within-task dependence and evolution across rounds.

A.1 Auxiliary Lemma and its Proof

For $U_i \in D$ and $B_b \in \mathcal{B}_{\text{val}}$, write the task-level gain destruction as:

$$\delta_{b,i}(H) := r_{B_{\text{rel}}}(\tau_i^{H, B_{\text{rel}}}, U_i) - r_{B_{\text{rel}}}(\tau_i^{H_0, B_{\text{rel}}}, U_i) - r_{B_b}(\tau_i^{H, B_b}, U_i) + r_{B_b}(\tau_i^{H_0, B_b}, U_i) \in [-2, 2].$$

For $D \in \{D_{\text{evo}}, D_{\text{disc}}\}$, let σ_i be independent Rademacher signs and define:

$$\mathfrak{R}_D := \mathbb{E} \max \left\{ \sup_H \left| \frac{1}{|D|} \sum_{U_i \in D} \sigma_i r_{B_{\text{rel}}}(\tau_i^{H, B_{\text{rel}}}, U_i) \right|, \sup_H \sup_{B_b \in \mathcal{B}_{\text{val}}} \left| \frac{1}{|D|} \sum_{U_i \in D} \sigma_i \delta_{b,i}(H) \right| \right\},$$

where the expectation is over the task-level evaluations and the Rademacher signs.

Lemma 1 Under Condition S1, define:

$$r := \max_{D \in \{D_{\text{evo}}, D_{\text{disc}}\}} \left\{ 2\mathfrak{R}_D + \sqrt{\frac{8 \log(8/\alpha)}{|D|}} \right\}, \quad x_t := \sqrt{\frac{8 \log(8T/\alpha)}{|D_{\text{conf},t}|}}. \quad (\text{A.1})$$

Then, with probability at least $1 - \alpha/2$, simultaneously,

$$\begin{aligned} \sup_H \left| \widehat{R}_{B_{\text{rel}}, D_{\text{evo}}}(\mathbf{A}, H) - R_{\text{rel}}(\mathbf{A}, H) \right| &\leq r, \\ \sup_H \sup_{\substack{B_b \in \mathcal{B}_{\text{val}} \\ D \in \{D_{\text{evo}}, D_{\text{disc}}\}}} \left| \widehat{\Delta}_{b,D}(H; H_0) - \Delta_b(H; H_0) \right| &\leq r, \\ \left| \widehat{\Delta}_{b_t, D_{\text{conf},t}}(H_t; H_0) - \Delta_{b_t}(H_t; H_0) \right| &\leq x_t \end{aligned} \quad (\text{A.2})$$

for every valid tested round t .

Proof. For either search set D with $n = |D|$, standard symmetrization (Zhang, 2023) and the bounded-differences inequality give:

$$\Pr \left\{ \max \left[\sup_H \left| \widehat{R}_{B_{\text{rel}}, D}(\mathbf{A}, H) - R_{\text{rel}}(\mathbf{A}, H) \right|, \sup_H \sup_{B_b \in \mathcal{B}_{\text{val}}} \left| \widehat{\Delta}_{b, D}(H; H_0) - \Delta_b(H; H_0) \right| \right] > 2\mathfrak{R}_D + z \right\} \leq e^{-nz^2/8},$$

since all task-level quantities above lie in $[-2, 2]$. Taking $z = \sqrt{8 \log(8/\alpha)/n}$ and a union bound over D_{evo} and D_{disc} gives total failure probability at most $\alpha/4$.

For each round, conditional on $\mathcal{F}_{D_{\text{conf}, t}}^-$, the confirmation contrast is an average of independent $[-2, 2]$ variables with mean $\Delta_{b_t}(H_t; H_0)$. Hence, Hoeffding's inequality gives:

$$\Pr \left\{ \left| \widehat{\Delta}_{b_t, D_{\text{conf}, t}}(H_t; H_0) - \Delta_{b_t}(H_t; H_0) \right| > x_t \mid \mathcal{F}_{D_{\text{conf}, t}}^- \right\} \leq 2e^{-|D_{\text{conf}, t}|x_t^2/8} = \frac{\alpha}{4T}.$$

Taking expectations and a union bound over $t = 1, \dots, T$ contributes at most another $\alpha/4$.

A.2 Proof of Theorem 1

Let $K = K(\mathcal{A}) < \infty$. By definition, there exist $B_{b_1}, \dots, B_{b_K} \in \mathcal{A}$ such that $B_{b_K \circ \dots \circ b_1} = B_0$. Closure of $\mathcal{B}_{\text{val}}$ gives $B_{b_j \circ \dots \circ b_1} \in \mathcal{B}_{\text{val}}$ for every $j = 1, \dots, K$.

For any H and $B_b, B_{b'} \in \mathcal{B}_{\text{val}}$, the definition of ρ gives:

$$\begin{aligned} \Delta_{b' \circ b}(H; H_0) - \Delta_b(H; H_0) - \Delta_{b'}(H; H_0) \\ \leq [\Delta_{b' \circ b}(H; H_0) - \Delta_b(H; H_0) - \Delta_{b'}(H; H_0)]_+ \leq \rho. \end{aligned}$$

Taking $b = b_{j-1} \circ \dots \circ b_1$ and $b' = b_j$ therefore yields, for $j = 2, \dots, K$,

$$\Delta_{b_j \circ \dots \circ b_1}(H; H_0) \leq \Delta_{b_{j-1} \circ \dots \circ b_1}(H; H_0) + \Delta_{b_j}(H; H_0) + \rho.$$

The base case is:

$$\Delta_{b_1}(H; H_0) \leq \Gamma_{\mathcal{A}}(H; H_0).$$

If

$$\Delta_{b_{j-1} \circ \dots \circ b_1}(H; H_0) \leq (j-1)\Gamma_{\mathcal{A}}(H; H_0) + (j-2)\rho,$$

then the preceding recurrence and $B_{b_j} \in \mathcal{A}$ give:

$$\begin{aligned} \Delta_{b_j \circ \dots \circ b_1}(H; H_0) &\leq (j-1)\Gamma_{\mathcal{A}}(H; H_0) + (j-2)\rho + \Gamma_{\mathcal{A}}(H; H_0) + \rho \\ &= j\Gamma_{\mathcal{A}}(H; H_0) + (j-1)\rho. \end{aligned}$$

Thus induction and $B_{b_K \circ \dots \circ b_1} = B_0$ give:

$$\Delta_{\text{BS}}(H; H_0) \leq K\Gamma_{\mathcal{A}}(H; H_0) + (K-1)\rho. \quad (\text{A.3})$$

Since $G_{\text{min}, \mathcal{A}} = G_{\text{rel}} - \Gamma_{\mathcal{A}}$ and $G_0 = G_{\text{rel}} - \Delta_{\text{BS}}$, the preceding bound also gives:

$$G_0(H; H_0) \geq G_{\text{min}, \mathcal{A}}(H; H_0) - (K-1)\{\Gamma_{\mathcal{A}}(H; H_0) + \rho\}.$$

For certification, condition on $\mathcal{F}_{D_{\text{cert}}}^-$. Then H^* and $\mathcal{A}$ are fixed. Let $m = |\mathcal{A}|$ and $n = |D_{\text{cert}}|$. For each $B_b \in \mathcal{A}$, the task-level summands defining $\widehat{G}_{b, D_{\text{cert}}}(H^*; H_0)$ are

$$r_{B_b}(\tau_i^{H^*, B_b}, U_i) - r_{B_b}(\tau_i^{H_0, B_b}, U_i) \in [-1, 1],$$

while those defining $\widehat{\Delta}_{b, D_{\text{cert}}}(H^*; H_0)$ are

$$r_{B_{\text{rel}}}(\tau_i^{H^*, B_{\text{rel}}}, U_i) - r_{B_{\text{rel}}}(\tau_i^{H_0, B_{\text{rel}}}, U_i) - r_{B_b}(\tau_i^{H^*, B_b}, U_i) + r_{B_b}(\tau_i^{H_0, B_b}, U_i) \in [-2, 2].$$

By the sampling conditions, these summands are conditionally independent across tasks and have conditional means $G_b(H^*; H_0)$ and $\Delta_b(H^*; H_0)$, respectively. Hence the one-sided Hoeffding inequalities give:

$$\begin{aligned}\Pr\left\{G_b(H^*; H_0) < \widehat{G}_{b, D_{\text{cert}}}(H^*; H_0) - s_G \mid \mathcal{F}_{D_{\text{cert}}}^-\right\} &\leq e^{-ns_G^2/2}, \\ \Pr\left\{\Delta_b(H^*; H_0) > \widehat{\Delta}_{b, D_{\text{cert}}}(H^*; H_0) + s_\Delta \mid \mathcal{F}_{D_{\text{cert}}}^-\right\} &\leq e^{-ns_\Delta^2/8},\end{aligned}$$

where we set $s_G = \sqrt{\frac{2 \log(4m/\alpha)}{n}}$, $s_\Delta = \sqrt{\frac{8 \log(4m/\alpha)}{n}}$. A union bound over all $B_b \in \mathcal{A}$ then gives, with conditional probability at least $1 - \alpha/2$,

$$G_b(H^*; H_0) \geq \widehat{G}_{b, D_{\text{cert}}}(H^*; H_0) - s_G, \quad \Delta_b(H^*; H_0) \leq \widehat{\Delta}_{b, D_{\text{cert}}}(H^*; H_0) + s_\Delta,$$

simultaneously for all $B_b \in \mathcal{A}$. Therefore,

$$G_{\min, \mathcal{A}}(H^*; H_0) \geq \widehat{G}_{\min, \mathcal{A}, D_{\text{cert}}}(H^*; H_0) - s_G, \quad \Gamma_{\mathcal{A}}(H^*; H_0) \leq \widehat{\Gamma}_{\mathcal{A}, D_{\text{cert}}}(H^*; H_0) + s_\Delta.$$

Substituting these bounds into equation A.3 and the corresponding lower bound for G_0 gives the two inequalities in Theorem 1. The same probability bound holds unconditionally by the tower property.

A.3 Proof of Theorem 2

Work on the common event $\mathcal{E}$ of Lemma 1. At every admitted round,

$$\Delta_{b_t}(H_t; H_0) \geq \widehat{\Delta}_{b_t, D_{\text{conf}, t}}(H_t; H_0) - x_t \geq \varepsilon + \gamma. \quad (\text{A.4})$$

Let $t_1 < \dots < t_m$ list the admitted rounds. If $m = 0$, the claim is immediate. For $j < k$, archive nesting gives $B_{b_{t_j}} \in \mathcal{A}_{t_k-1}$. Empirical feasibility at the later round and the uniform evolution bound give:

$$\Delta_{b_{t_j}}(H_{t_k}; H_0) \leq \widehat{\Delta}_{b_{t_j}, D_{\text{evo}}}(H_{t_k}; H_0) + r \leq \varepsilon + r. \quad (\text{A.5})$$

Combining this inequality with confirmation at the earlier round yields:

$$\sup_{B_b \in \mathcal{B}_{\text{val}}} |\Delta_b(H_{t_j}; H_0) - \Delta_b(H_{t_k}; H_0)| \geq \Delta_{b_{t_j}}(H_{t_j}; H_0) - \Delta_{b_{t_j}}(H_{t_k}; H_0) \geq \gamma - r.$$

Because $0 < \gamma - r \leq 1$, the original packing condition applies and gives $m \leq (C/(\gamma - r))^d$. If the round budget exceeds this bound and a valid proposal is tested every round, at least one round is not confirmed.

A.4 Proof of Theorem 3

Work on the event $\mathcal{E}$ of Lemma 1. If $\text{Conf}_t = 0$, then:

$$\Delta_{b_t}(H_t; H_0) \leq \widehat{\Delta}_{b_t, D_{\text{conf}, t}}(H_t; H_0) + x_t < \varepsilon + \gamma + 2x_t.$$

By empirical Challenger optimality and the uniform discovery bound,

$$\begin{aligned}\sup_{B_b \in \mathcal{B}_{\text{val}}} \Delta_b(H_t; H_0) &\leq \sup_{B_b \in \mathcal{B}_{\text{val}}} \widehat{\Delta}_{b, D_{\text{disc}}}(H_t; H_0) + r \\ &= \widehat{\Delta}_{b_t, D_{\text{disc}}}(H_t; H_0) + r \\ &\leq \Delta_{b_t}(H_t; H_0) + 2r < \varepsilon + \gamma + 2x_t + 2r.\end{aligned}$$

Now take any $\tilde{H}$ satisfying $\Delta_b(\tilde{H}; H_0) \leq \varepsilon - r$ for all $B_b \in \mathcal{B}_{\text{val}}$. Since $\mathcal{A}_{t-1} \subseteq \mathcal{B}_{\text{val}}$,

$$\widehat{\Delta}_{b, D_{\text{evo}}}(\tilde{H}; H_0) \leq \Delta_b(\tilde{H}; H_0) + r \leq \varepsilon, \quad B_b \in \mathcal{A}_{t-1},$$

so $\tilde{H}$ is feasible for the empirical Proposer. Hence,

$$\begin{aligned}R_{\text{rel}}(\mathbf{A}, H_t) &\geq \widehat{R}_{B_{\text{rel}}, D_{\text{evo}}}(\mathbf{A}, H_t) - r \\ &\geq \widehat{R}_{B_{\text{rel}}, D_{\text{evo}}}(\mathbf{A}, \tilde{H}) - r \\ &\geq R_{\text{rel}}(\mathbf{A}, \tilde{H}) - 2r.\end{aligned}$$

Subtracting $R_{\text{rel}}(\mathbf{A}, H_0)$ and taking the supremum over $\tilde{H}$ gives the second claim. If $\varepsilon \geq r$, the comparison set contains H_0 .

B Method Implementation Details

Section B.1 defines the comparison methods and shared settings. Section B.2 describes the Proposer and Challenger updates. Section B.3 specifies the counterfactual transformations and validity firewall. Section B.4 summarizes the target-agent, Proposer, and Challenger harnesses.

B.1 Comparison Methods and Shared Settings

Within OfficeQA, all methods use the same foundation model and reasoning setting, initial harness H_0 , request schema, sampling parameters, corpus, retrieval indexes, tools, context policy, step limit, time limit, retry policy, and scorer. In the primary configuration, all methods use gpt-5.6-sol with high reasoning effort for every model role, including the target agent, Proposer, and Challenger. The original HarnessCompass configuration uses gpt-5.4 in the non-thinking setting (Zhang et al., 2026). We use a reasoning-enabled Proposer in OfficeQA so that any observed shortcut behavior cannot be simply attributed to insufficient reasoning. For Syn-Ledger, we instead use gpt-5.6-sol in the non-thinking setting for all model calls to reduce experimental cost. The agent A is not given its data-role assignment. Table B.1 compares the four comparison methods. Each optimized method runs three Proposer rounds. The fixed Generalization Gate applies to HarnessCompass and B_0 -Access; CHASE uses archive constraints without that Gate.

Table B.1: Method comparison in this study. “Shortcut-aware” indicates that harness evolution explicitly accounts for shortcut behavior; “ B_0 -free” indicates no access to B_0 during optimization.

Method	Proposer	Challenger	Shortcut-aware	B_0 -free
RawHarness	✗	✗	✗	✓
HarnessCompass	✓	✗	✓	✓
B_0 -Access	✓	✗	✓	✗
CHASE	✓	✓	✓	✓

RawHarness keeps H_0 unchanged to measure gains from optimization. HarnessCompass performs released-score optimization with the fixed Generalization Gate, following Zhang et al. (2026). B_0 -Access evaluates Proposer candidates directly on the neutralized benchmark and selects by $\hat{R}_{B_0, D_{\text{evo}}}$, testing access to B_0 during optimization. The custodian exposes the B_0 task interface to this comparator while retaining answers and generation metadata. For cost control, we use two rollouts per harness-question pair on D_{evo} , two on D_{disc} , five on each $D_{\text{conf}, t}$, and three on D_{cert} .

B.2 Proposer and Challenger Updates

For controlled comparison, CHASE uses the same Proposer backbone as HarnessCompass, including its feedback elicitation and separate structural and guidance tracks (Zhang et al., 2026). For HarnessCompass and B_0 -Access, the Generalization Gate is specified once in the system prompt, fixed before optimization, and applied to every candidate edit. CHASE does not use this Gate.

At round t , HarnessCompass and CHASE each begin with their current harness H_{t-1} and evaluate it on the full D_{evo} . Following the history interface introduced by Meta-Harness (Lee et al., 2026), the Proposer can then inspect the method’s complete optimization history through a controlled read-only file interface. This history contains the code of the current harness and all previously evaluated candidates, together with their scores and execution records, including prompts, tool calls, model outputs, and state changes. The files retain their original names and relative paths so that the Proposer can navigate the history directly. HarnessCompass and CHASE have the same access to their own histories, but neither method can inspect the other method’s runs. To reduce evaluation cost, we reuse previously computed estimates.

Proposer Update. Following HarnessCompass, the Proposer analyzes this history through two complementary feedback passes (Zhang et al., 2026). Proactive feedback describes how the current harness affects the agent’s behavior and suggests possible improvements, while hindsight feedback uses the observed outcomes to identify behaviors associated with success or failure. The Proposer then develops one candidate for structural components and another for guidance components. Finally, it applies R^3 —Revision, Recombination, and Refinement—to revise the two candidates and combine compatible changes into a third, integrated candidate (Zhang et al., 2026).

HarnessCompass selects a new harness only when one of the three candidates achieves a higher released-benchmark score; otherwise, it retains the current harness. In CHASE, before candidate generation, the Proposer is also given the current archive $\mathcal{A}_{t-1}$ and the corresponding D_{evo} scores and trajectories. The archive constraints are included in the Proposer prompt during candidate generation and are also enforced during candidate selection. After evaluation, CHASE first removes every candidate that violates: $\hat{\Delta}_{b, D_{\text{evo}}}(H; H_0) \leq 0.05$, for every $B_b \in \mathcal{A}_{t-1}$. Among the newly generated candidates that satisfy the archive constraints, CHASE selects the one with the highest released-benchmark score, but adopts a new harness only if its score is at least as high as that of H_{t-1} . Otherwise, it retains H_{t-1} if it remains archive-feasible, and falls back to H_0 if not. Thus, each round optimizes released-benchmark performance subject to the counterfactual constraints accumulated in earlier rounds, while always retaining a feasible reference harness.

Challenger Update. After CHASE selects H_t , the Challenger searches for benchmark transformations Φ_b under which the gain of H_t over H_0 may disappear. In each round, it initially generates eight proposals, each containing a natural-language description and a typed transformation specification. On OfficeQA, these specifications modify protocol-level features while preserving the underlying question and answer; see Supplementary B.3 for details.

Before any target-agent rollout, the host applies the validity firewall and duplicate screen to all eight proposals. A proposal is discarded if it fails any validity check in Table B.2 or repeats a transformation already stored in the archive. To detect such repeats, the host represents each transformation as an ordered sequence of allowlisted API operators and their normalized arguments. A new proposal is treated as a duplicate when this representation matches that of an archived transformation. A hash of the representation is stored with each archived transformation and used for subsequent comparisons.

The host then ranks the proposals that pass both screens lexicographically by validator-derived fixture change, activation coverage, vulnerability-map coverage, and archive novelty, using proposal order only to break any remaining ties. It retains the highest-ranked proposal and, whenever possible, the highest-ranked proposal from a different transformation family, so that at most two proposals proceed to evaluation on D_{disc} . After evaluation on the full D_{disc} , CHASE selects the transformation with the largest estimated gain destruction. If no admissible transformation is obtained in that round, the archive remains unchanged.

Since the Challenger searches over D_{disc} , a large value of $\hat{\Delta}_{b, D_{\text{disc}}}(H_t; H_0)$ may be inflated by selection. For OfficeQA, the selected transformation is first frozen and evaluated on a 24-question set $D_{\text{conf}, t}$, with five rollouts per question. We evaluate both H_t and H_0 under the released benchmark B_{rel} and the proposed counterfactual benchmark B_{b_t} . This requires the following evaluation: 2 harnesses $\times$ 2 benchmarks $\times$ 24 questions $\times$ 5 rollouts = 480 confirmation runs. The proposal enters the archive when the gain destruction pooled over the $24 \times 5 = 120$ question-rollout comparisons is at least $\eta = 0.075$. After three rounds, CHASE selects its final harness from $H_0, \dots, H_3$ using the archive-feasibility rule in Section 3, with $\varepsilon = 0.05$. For fixed task and rollout budgets, the cost of evaluating each harness scales linearly with the number of archived benchmarks $|\mathcal{A}_{t-1}|$.

B.3 Counterfactual Transformations and Validity Firewall

OfficeQA permits reversible changes to identifiers, document layout, tool-interface schemas, and enumeration order, including valid compositions. For Syn-Ledger, the Challenger proposes transformations within the permitted protocol surface; the reference neutralizers and placebos in Section D.2 serve only as construction-audit tools, not Challenger inputs. Across both benchmarks, transformations preserve task labels and available information (Ribeiro et al., 2020).

Table B.2: Validity firewall: every check must pass.

Invariant	Required check
Task and answer	The question, requested target, and correct answer are unchanged.
Available information	Document contents and membership match after undoing identifier mappings; no content is added, removed, altered, or moved between documents.
Access and resources	The same documents and tool operations remain accessible, with unchanged tool-call, context, and step limits.
Scoring	Answer normalization, numeric tolerance, and scorer verdicts on fixed correct and incorrect outputs are unchanged.
Declared changes	Only the protocol metadata declared in the proposal are modified.
Task independence	Rules do not depend on questions, answers, gold sources, or dataset splits.
Replay and inversion	Identical inputs and settings reproduce the transformation; reversible changes recover the original state.

Table B.2 lists the seven checks evaluated by the host-side validity firewall. Each check returns a Boolean value, and a proposal is accepted only when all seven values are true. The Challenger’s natural-language validity claims are retained in the audit record but are not used as substitutes for these executable checks.

Table B.3: Host-side pseudocode for the Syn-Ledger validity firewall.

Syn-Ledger validity firewall: host-side pseudocode.

- 1 Parse the Challenger output as a typed protocol specification s ; reject any operator, argument, or modified field outside the allowlisted protocol API.
- 2 Canonicalize s into an ordered operator–argument representation. Reject s if the same canonical representation already occurs in the archive.
- 3 For every firewall task U , execute $Z_1 \leftarrow \text{Render}(U, s, \text{seed})$ and $Z_2 \leftarrow \text{Render}(U, s, \text{seed})$, and parse Z_1 back into typed document records.
- 4 *Task and answer*: verify that the question and requested target are unchanged and that recomputing the answer from the parsed records returns the original correct answer.
- 5 *Available information*: verify equality of document membership and of the canonical record multisets before and after transformation.
- 6 *Access, resources, and scoring*: verify equality of the accessible tool operations, resource limits, scorer hash, and scorer verdicts on fixed correct and incorrect output fixtures.
- 7 *Declared changes and task independence*: compute the modified fields and require them to be a subset of the fields declared by s ; reject any reference to a question identifier, answer, gold source, or dataset split.
- 8 *Replay and inversion*: require $Z_1 = Z_2$ and require the canonical records recovered from Z_1 to equal those of U .
- 9 Return PASS if and only if all seven checks in Table B.2 pass; otherwise return the failed check names and reject the proposal before D_{disc} evaluation.

Table B.3 gives the Syn-Ledger implementation of the checks in Table B.2. The canonical parser reconstructs the typed records from each rendered document, after which the benchmark generator independently recomputes the correct answer. Identity, valid single-field transformations, and valid compositions serve as positive controls; transformations that modify the answer, introduce undeclared task-dependent fields, or use a non-allowlisted operator serve as negative controls. OfficeQA applies the same pass-all interface with benchmark-specific JSON canonicalization and tool-output fixtures; we omit the analogous dispatcher code.

OfficeQA firewall example. In the first OfficeQA round, the Challenger first returned eight typed transformation proposals. The host-side firewall accepted seven and rejected one before any target-agent rollout on D_{disc} . The rejected proposal specified a seeded permutation of the search results. When executed on the validity firewall, however, it left the result order unchanged, so the firewall rejected it as an inactive transformation and removed it from the candidate pool. The excluded proposal was inactive, rather than shown to violate semantic validity.

B.4 Initial Target-agent Harness, Proposer Harness, and Challenger Harness

Tables B.4–B.6 summarize the three harness designs, including their accessible context, tools, control, outputs, and condensed core instructions. The target-agent harness evolves from H_0 ; the Proposer and Challenger use fixed harness configurations. The instructions below are illustrative templates; benchmark-specific tool schemas and run histories are supplied separately.

Table B.4: Initial target-agent harness H_0 .

Accessibility	OfficeQA: the evaluated release’s full transformed-text corpus. Syn-Ledger: the current task’s twelve documents. No gold annotations or generation metadata.
Memory	No task-specific long-term memory.
Tools / retrieval	OfficeQA: content-only passage BM25; conjunctive all matching on the unmodified query, with native BM25 ranking. Syn-Ledger: task-local tools and result order (Section D.1).
Control	Agent-directed tool use within the shared budget; no harness-side query rewriting, fallback retrieval, or reranking.
Core prompt. <i>Use the available tools to answer the question and return the answer in the required format.</i>	

Table B.5: Proposer harness: generating target-agent harness candidates.

Accessibility	Harness code, scores, and complete D_{evo} histories through read-only files; CHASE also receives $\mathcal{A}_{t-1}$ and its D_{evo} feedback.
Edit target	Target-agent prompts, memory, retrieval, tool wrappers, and control code; the target model, corpus, available tool operations, scorer, and budget stay fixed.
Control	Diagnose failures and regression risks; develop structural and guidance candidates, then integrate compatible changes through R^3 (Section B.2).
Output	Executable edits, targeted failures, activation and stopping conditions, expected tool cost, and regression risks.
Selection	HarnessCompass: released score with the fixed Gate. B_0 - Access: $\hat{R}_{B_0, D_{\text{evo}}}$ with that Gate. CHASE: released score with $\hat{\Delta}_{b, D_{\text{evo}}}(H; H_0) \leq \varepsilon$ for every $B_b \in \mathcal{A}_{t-1}$.
Core prompt. <i>Improve accuracy on D_{evo}. Inspect every task–rollout result; distinguish stable successes, unstable outcomes, and stable failures. Separate observations from hypotheses. Use distinct mechanisms and concise conditional guidance; preserve behavior outside each edit’s activation condition. Do not hard-code question identifiers, dataset membership, answers, or source filenames learned as labels.</i>	

Table B.6: Challenger harness: generating benchmark transformations.

Accessibility	H_t , H_0 , the archive, and permitted paired scores and trajectories on D_{disc} through read-only history files.
Edit target	Benchmark protocol changes from Section B.3; both H_t and H_0 remain unchanged.
Control	Generate distinct proposals. The evaluator rejects invalid or archived duplicates, ranks valid proposals on D_{disc} , and freezes the selected one for fresh confirmation.
Output	Typed transformation specification, declared changes, suspected shortcut, supporting traces, and replay settings.
Core prompt. <i>Maximize $\hat{\Delta}_{b, D_{\text{disc}}}(H_t; H_0)$: destroy gain over H_0, not merely raw score. Preserve questions, answers, document contents and membership, tool access, budgets, and scorer. Apply the same question-independent rule to both harnesses.</i>	

C Additional Results of OfficeQA

Section C.1 describes corpus preprocessing and data allocation, and Section C.2 specifies evaluation and reporting. Section C.3 presents the OfficeQA counterexample and its implementation. Section C.4 details the HarnessCompass Generalization Gate used in OfficeQA, and Section C.5 reports token use and wall time during search and certification.

C.1 Preprocessing and Task Allocation

OfficeQA Full contains 246 questions², which use the same corpus of 697 documents from the U.S. Treasury Bulletin collection spanning 1939–2025 (Opsahl-Ong et al., 2026; Databricks, 2026). Each question record includes an identifier, question, answer, source URLs, corresponding source files, and difficulty label. The corpus is released as original PDFs, parsed JSON, and transformed text with tables represented in Markdown. We use the transformed-text corpus throughout; the original PDFs and parsed JSON are not used. Question identifiers and texts are unique within release.

We examine the question texts and corpus layout to quantify the formatting regularity described in Section 1 (Table C.1). A question mentions a numerical scale if it contains the whole word *thousand*, *million*, *billion*, or *trillion*, allowing plural forms and ignoring case. In the corpus, we count tables with at least two columns, a header, a delimiter row, and a data row. This yields 94,303 tables, including contents tables, repeated tables across bulletin editions, and separately rendered parts of longer tables. We identify unit statements by matching non-table lines containing explicit numerical-scale or unit expressions involving hundreds, thousands, millions, billions, trillions, dollar(s), cent(s), percent/percentage(s), basis points, ounces, or units. Matching ignores case and allows leading heading markers, an opening bracket, and an Amounts, Figures, or Dollar Amounts prefix, optionally preceded by All. This rule identifies 66,951 unit statements. A Note/Source line begins with either label and a colon, period, or dash, allowing plurals and leading heading markers. Adjacency ignores blank lines only.

Table C.1: Descriptive layout regularities in OfficeQA Full and its 697-document text corpus.

Criterion	Count / denominator	Percentage
Questions mentioning numerical scales	143 / 246	58.1%
Identified unit statements immediately before a table	63,747 / 66,951	95.2%
Table blocks preceded by an identified unit statement	63,747 / 94,303	67.6%
Table blocks followed by a Note/Source line	25,253 / 94,303	26.8%

All 697 documents contain examples of units before tables and Note/Source lines after tables. These counts describe marked text and its position; notes may apply to a table group or an entire bulletin. For a concrete example, Table FFO-7 in the April 1980 bulletin places its million-dollar unit above the table and explains below it that interfund payments are excluded when calculating trust-fund receipts and outlays. Reading near the table boundaries can therefore supply both the numerical scale and an aggregation rule. The question count records explicit scale mentions; it does not identify which questions require a particular adjacent note.

OfficeQA Pro V2 contains 90 questions over a separate corpus of 1,435 parsed documents (Databricks, 2026)³. We apply the released conversion functions to all parsed inputs and freeze the resulting text before evaluation. The conversion does not load questions, answers, gold sources, or model outputs. Pro V2 results are analyzed separately from the primary OfficeQA experiment.

For each question, we collect all annotated source filenames, including those listed directly in the record and those recovered from its official source URLs. This yields 296 unique source filenames across the 246 questions in OfficeQA Full. We use shared source files to group related questions into 100 source components. The questions are then divided into D_{evo} (49 questions), D_{disc} (49 questions), three round-specific confirmation sets $D_{\text{conf},t}$ (24 questions each), and D_{cert} (76 questions). The source components assigned to D_{evo} and D_{cert} do not appear in any other split. By design, D_{disc} and the confirmation sets may share source components: each confirmation set contains 11 questions from components that also appear in D_{disc} and 13 questions from components that do not appear in D_{disc} . Questions never repeat across splits. Each confirmation set contains 11 easy and 13 hard questions. The allocation uses only question identifiers, difficulty labels, and source membership, without using model outcomes. The certification set D_{cert} is accessed only after the final harnesses, $\mathcal{A}_3$, and the analysis code are fixed.

²OfficeQA Full: <https://huggingface.co/datasets/databricks/officeqa>

³OfficeQA Pro V2: <https://huggingface.co/datasets/databricks/officeqa-pro-v2>

A purely random question-level split would ignore the fact that multiple OfficeQA questions can rely on the same source files, making the comparison sensitive to accidental source overlap. We therefore use source components to control how related questions are distributed. We test whether a discovered benchmark-wide shortcut reproduces on new questions involving related sources, and whether the same shortcut extends beyond the sources used for discovery.

C.2 Evaluation and Reporting

We compute all OfficeQA correctness indicators using the released `fuzzy_match_answer` function at 0% numerical tolerance, following the scoring procedure in Alzubi et al. (2026). The same answer normalization and scoring rule are held fixed across B_{rel} and every counterfactual benchmark B_b . Gold source annotations are not exposed to the target agent and do not enter the score.

The primary certification matrix evaluates the final harness H from each method in the main comparison—RawHarness, HarnessCompass, and CHASE—on the 76 certification questions under every benchmark in $\mathcal{A}_3$. The archive contains B_{rel} and the counterfactuals confirmed during CHASE optimization. This matrix contains $3 \times 76 \times |\mathcal{A}_3|$ method–question–benchmark combinations, with three rollouts per combination. Rollout scores are averaged within question and then across questions. OfficeQA Pro V2 is evaluated separately under its released protocol, with three rollouts per method–question pair; its score is denoted by $\hat{R}_{\text{ProV2}}$. We also report the archive summaries $\hat{R}_{\text{avg}, \mathcal{A}_3}$ and $\hat{R}_{\text{min}, \mathcal{A}_3}$, using the shorthand defined in the main text.

C.3 Tracing Counterfactual Harness Search and Evolution

We examine H_1 , B_{b_1} , and the harness candidates proposed in rounds 2–3 of the OfficeQA experiment.

The first-round Proposer and a possible shortcut. The initial harness H_0 is specified in Table B.4. Table C.2 reports the changes in H_1 . Its tool-call budget remain unchanged from H_0 . H_1 only adds model instructions and a conditional search-query rewrite.

Table C.2: Changes from H_0 to H_1 .

Component	Changes from H_0
Prompt and memory	Adds reminders for recovery and numerical checking. Once the needed values are found, the model is encouraged to verify or calculate rather than continue searching.
Conditional guidance	Adds instructions for tracking units, converting scales, preserving numerical precision, and returning answers in the required order and format.
Search-query rewrite	When at least four tool calls remain, removes stopwords, keeps up to eight terms of at least three characters, and changes search from requiring all terms to allowing any term.
Tool-use guidance	Tells the model when to retry, to reserve two reads, and to stop once sufficient evidence has been found.

H_1 broadens retrieval by allowing any query term to match and encourages the model to verify or calculate once the needed values appear to have been found. Its numerical guidance emphasizes units and precision, but does not explicitly tie each value to its table context, such as headers or nearby notes. We therefore hypothesize that the regular placement of such context in OfficeQA may itself signal that enough evidence has been found. Relocating that context could then change which values the model uses or whether it continues reading.

The first-round Challenger and the counterfactual. The Challenger returns a natural-language proposal together with a typed transformation specification. As illustrated in Figure C.1, the proposal is to collect a table’s title, unit line, footnotes, source notes, scope explanations, and surrounding explanatory prose before the table, targeting H_1 ’s possible reliance on their customary locations.

For evaluation, we execute the Challenger-generated transformation Φ_{b_1} on the agent-visible JSON search results. For each result containing a `text` field, the code moves the complete string into `table_context.content`, sets `table_context.placement` to "before", and removes the original `text` field. The string’s internal order, document identifier, and line references are preserved; no table parsing or within-string reordering is performed. This transformation applies to every text-bearing search result, changing its field names and nesting while leaving the underlying documents, result counts, read limits, tool-call budgets, questions, and scoring rule unchanged.

On D_{disc} , the gain destruction under Φ_{b_1} is 7.14%, compared with 4.08% for the other candidate transformation, a ten-line passage-boundary shift, which moves each eligible internal passage boundary ten lines later while preserving

TABLE IFS-1.—U.S. Reserve Assets

[In millions of dollars. Source: Treasury International Capital Reporting System]

End of calendar year or month	Total reserve assets ¹ (1)	Gold stock ² (2)
2006	65,895	11,041
2007	70,565	11,041
2008	77,648	11,041
2009	130,760	11,041

¹ Beginning July 1974, the International Monetary Fund (IMF) adopted a technique for valuing the special drawing right (SDR) based on a weighted average of exchange rates for the currencies of selected member countries. The U.S. SDR holdings and reserve position in the IMF also are valued on this basis beginning July 1974.

² Treasury values its gold stock at \$42.2222 per fine troy ounce and pursuant to 31 United States Code 5117 (b) issues gold certificates to the Federal Reserve at the same rate against all gold held.

TABLE ESF-2.—Income and Expense

[In thousands of dollars. Source: Office of the Assistant Secretary of the Treasury for Management]

	Current quarter Jan. 1, 2011, through Mar. 31, 2011
Income and expense	
Profit (+) or loss (-) on:	
Foreign exchange	603,216
Adjustment for change in valuation of SDR holdings and allocations ¹	
	75,671

¹ Beginning July 1974, the International Monetary Fund adopted a technique for valuing the SDRs based on a weighted average of exchange rates for the currencies of selected member countries. The U.S. SDR holdings and allocations are valued on this basis beginning July 1974.

Note. — Annual balance sheets for fiscal years 1934 through 1940 appeared in the 1940 "Annual Report of the Secretary of the Treasury" and those for succeeding years appeared in subsequent reports through 1980. Quarterly balance sheets beginning with December 31, 1938, have been published in the "Treasury Bulletin." Data from inception to September 30, 1978, may be found on the statements published in the January 1979 "Treasury Bulletin."

TABLE IFS-1.—U.S. Reserve Assets

[In millions of dollars. Source: Treasury International Capital Reporting System]

End of calendar year or month	Total reserve assets ¹ (1)	Gold stock ² (2)
2006	65,895	11,041
2007	70,565	11,041
2008	77,648	11,041
2009	130,760	11,041

¹ Beginning July 1974, the International Monetary Fund (IMF) adopted a technique for valuing the special drawing right (SDR) based on a weighted average of exchange rates for the currencies of selected member countries. The U.S. SDR holdings and reserve position in the IMF also are valued on this basis beginning July 1974.

² Treasury values its gold stock at \$42.2222 per fine troy ounce and pursuant to 31 United States Code 5117 (b) issues gold certificates to the Federal Reserve at the same rate against all gold held.

TABLE ESF-2.—Income and Expense

[In thousands of dollars. Source: Office of the Assistant Secretary of the Treasury for Management]

	Current quarter Jan. 1, 2011, through Mar. 31, 2011
Income and expense	
Profit (+) or loss (-) on:	
Foreign exchange	603,216
Adjustment for change in valuation of SDR holdings and allocations ¹	
	75,671

¹ Beginning July 1974, the International Monetary Fund adopted a technique for valuing the SDRs based on a weighted average of exchange rates for the currencies of selected member countries. The U.S. SDR holdings and allocations are valued on this basis beginning July 1974.

Note. — Annual balance sheets for fiscal years 1934 through 1940 appeared in the 1940 "Annual Report of the Secretary of the Treasury" and those for succeeding years appeared in subsequent reports through 1980. Quarterly balance sheets beginning with December 31, 1938, have been published in the "Treasury Bulletin." Data from inception to September 30, 1978, may be found on the statements published in the January 1979 "Treasury Bulletin."

Φ_{b_1}



Figure C.1: Illustration of the Challenger's table-context relocation proposal on two OfficeQA excerpts from the September 2011 *Treasury Bulletin*: IFS-1 (top) and ESF-2 (bottom). Highlighted notes are shown above their tables to visualize the proposal, with their wording unchanged. The evaluated code re-encodes JSON search results as described in the text.

complete document coverage and the original line order. We therefore select Φ_{b_1} for evaluation on $D_{\text{conf},1}$. Across five rollouts for each of its 24 questions under B_{b_1} , the recorded gain destruction is 14%, exceeding the 7.5% confirmation threshold, and B_{b_1} is added to the archive.

Second-round candidates and fallback to H_0 . In the second round, the Proposer generates three revisions of H_1 (Table C.3). The guidance candidate most directly responds to this concern by instructing the model to track the table context of each value it uses. The structural and integrated candidates instead emphasize combining evidence across periods and documents. All three retain H_1 's prompt, memory, retrieval settings, and tool-use guidance.

For the selection analysis, we apply $\varepsilon = 0.05$. Each harness is evaluated on the same 49 evolution questions under both B_{rel} and B_{b_1} . We compute $\hat{\Delta}_{b_1} = \hat{G}_{\text{rel}} - \hat{G}_{b_1}$ and require $\hat{\Delta}_{b_1} \leq 0.05$. For both selections, the archive contains B_{rel} and B_{b_1} , so B_{b_1} provides the only nontrivial constraint, as shown in Table C.4. All three candidates and H_1 fail, giving $H_2 = H_0$. The structural and guidance candidates come closest, but each loses 6.12%. Since the second-round selection returned $H_2 = H_0$, CHASE skipped Challenger generation, discovery, and confirmation in this round.

Third-round candidates after failure feedback. The third-round Proposer starts from $H_2 = H_0$. Its input reports the second-round constraint failures and fallback, and explicitly states that a higher released score alone is insufficient. Table C.5 summarizes the candidates. All three retain H_0 's generic prompt and empty memory, rather than inheriting H_1 's instruction to stop repeated searching once the needed values appear to have been found.

The third round starts again from $H_2 = H_0$, but the Proposer now has feedback from both the first-round counterfactual and the second-round constraint failures. After the counterfactual exposes a loss of advantage and the

Table C.3: Candidate pool of the second-round Proposer.

Candidate	Main changes
Structural	Decompose multi-period queries, merge results in round-robin order, promote document diversity, and combine needed values across documents.
Guidance	Bind each value to its title, date, unit, row label, and column header. Read adjacent headers or footnotes before using totals, subtotals, or year-to-date columns.
Integrated	Track each needed value by its fiscal year, calendar year, publication date, period, and whether it is cumulative; add rules for numerical and multi-period questions.

Table C.4: Second-round selection at $\varepsilon = 0.05$.

Harness	$\hat{G}_{\text{rel}}$	$\hat{G}_{b_1}$	$\hat{\Delta}_{b_1}$	Pass
H_1	+8.16%	-5.10%	13.27%	No
Structural	+15.31%	+9.18%	6.12%	No
Guidance	+9.18%	+3.06%	6.12%	No
Integrated	+15.31%	-2.04%	17.35%	No

second-round revisions fail the archive constraint, the Proposer returns to H_0 while retaining feedback on these failures. The integrated candidate makes the clearest change to the earlier retrieval strategy: it preserves the query’s specificity and uses broad matching as a recovery step rather than at the outset. It combines this change with explicit numerical guidance, without inheriting H_1 ’s instruction to shift away from repeated searching once the needed values appear to have been found. The resulting design therefore revisits both how evidence is retrieved and how the model is instructed to proceed after finding values, rather than simply adding another reminder about table context. This sequence illustrates how counterfactual feedback can inform subsequent harness design, not merely reject candidates with high released scores.

As shown in Table C.6, the integrated candidate achieves the highest released score among the candidates and is therefore selected as H_3 . In contrast to the larger but less stable gains rejected in the second round, the third-round revision yields a smaller released gain that is more consistent with the archived counterfactual constraint.

Figure C.2 illustrates how the archived counterfactual shapes candidate selection. The second-round candidates achieve larger released gains but violate the archive constraint, whereas the third-round integrated candidate provides a smaller released gain within the same tolerance.

After selecting H_3 , the third-round Challenger evaluates two transformations on D_{disc} . A ten-line passage-boundary shift mentioned above produces a gain destruction of 10.20% and is selected for evaluation on $D_{\text{conf},3}$. Across five rollouts on each of its 24 questions, the recorded gain destruction is 4.08%, which is below the 7.5% confirmation threshold. The transformation is therefore not added to the archive.

C.4 HarnessCompass Generalization Gate on OfficeQA

Our HarnessCompass baseline retains the optimization components reported by Zhang et al. (2026): proactive and hindsight feedback, separate structural and guidance tracks, R^3 integration, a fixed Generalization Gate, and released-score-based updates. The shared OfficeQA settings for HarnessCompass and CHASE are specified in Section B.1.

HarnessCompass specifies its Generalization Gate once in the meta-agent system prompt and applies it uniformly to every candidate edit across its harness components. The Gate has two requirements. Its content requirement admits task-agnostic decision criteria with explicit applicability conditions and rejects edits tied to a particular task instance, test, private symbol, file path, or task-specific token. Its placement requirement keeps executable capability changes in structural components and behavioral guidance in the system prompt or memory. HarnessCompass evaluates this design on SWE-bench Verified but does not report an OfficeQA-specific Gate (Zhang et al., 2026).

For the OfficeQA comparison, we retain these two requirements and construct one OfficeQA-specific Generalization Gate before optimization. The Gate scans all six candidate surfaces used in our experiment: the system prompt, long-term memory, retrieval policy, tool policy, policy graph, and guidance modules. Across these surfaces, it admits only cross-question reusable rules with explicit applicability conditions. A deterministic scan rejects question identifiers and data-role labels, shared eight-token spans from observed questions, gold source filenames, and question-specific

Table C.5: Candidate pool of the third-round Proposer.

Candidate	Main changes
Structural	Instructs the model to search separately for different periods, combine search results, and assemble evidence across documents, without changing the underlying retrieval procedure.
Guidance	Adds conditional instructions to convert source values to the requested units, preserve full precision during calculation, and follow the requested rounding order and answer format.
Integrated	Combines these types of numerical guidance with revised retrieval: retain all query terms except stopwords, first require all retained terms to match, and allow matches on any term only when the initial search has low coverage.

Table C.6: Third-round selection at $\varepsilon = 0.05$.

Harness	$\hat{G}_{\text{rel}}$	$\hat{G}_{b_1}$	$\hat{\Delta}_{b_1}$	Pass
Structural	-1.02%	-1.02%	0.00%	Yes
Guidance	-2.04%	-6.12%	4.08%	Yes
Integrated	+4.08%	0.00%	4.08%	Yes

year-answer-number pairs. Structural candidates are also required to validate against the declared policy schema and change a declared retrieval or control-policy field, whereas advisory instructions are confined to the prompt, memory, or guidance modules. Every HarnessCompass candidate must pass these checks before target evaluation.

C.5 Token Use and Wall Time

We compare certification resource use using three rollouts per question. Across two benchmarks and 76 questions, this yields 456 evaluation runs per method. Table C.7 reports total token use, token use excluding cached input, and wall time. Wall time includes recorded model-invocation time and retry waits, while summed wall time aggregates these values across all runs and therefore does not represent elapsed clock time under parallel execution. Among the three methods, CHASE used the fewest uncached input plus output tokens: 22.39 million. Relative to HarnessCompass, CHASE used 17.2% fewer total tokens and 23.1% less summed wall time. Relative to RawHarness, it used 4.6% more total tokens and 8.3% more summed wall time. Its median wall time was 22.9 minutes per run, between RawHarness at 20.7 minutes and HarnessCompass at 24.5 minutes.

Table C.7: Resource use during certification with three rollouts per question. For per-run wall time, we report the median together with the interquartile range (IQR) in brackets, i.e., median [25th percentile, 75th percentile].

Method	Token use (millions)		Wall time	
	Total	Excluding cached input	Median [IQR] per run (minutes)	Sum (hours)
RawHarness	93.82	24.30	20.7 [16.5, 25.5]	166.70
HarnessCompass	118.50	24.96	24.5 [19.8, 31.8]	234.83
CHASE	98.15	22.39	22.9 [17.3, 28.5]	180.52

We report certification separately because RawHarness has no search stage, and the two search methods run different numbers and types of attempts. Table C.8 counts every search attempt with a recorded total token count, including attempts that were not retained. Compared with HarnessCompass, CHASE runs 2.47 times as many attempts and uses 2.43 times as many total tokens. This additional search includes Challenger discovery and confirmation.

D Additional Results of Synthetic Benchmark Syn-Ledger

Syn-Ledger contains 320 multi-document arithmetic tasks with controlled benchmark-wide shortcuts. Section D.1 describes task construction, the agent interface, and scoring; Section D.2 describes benchmark-wide shortcut construction and validation; Section D.3 gives the task allocation.

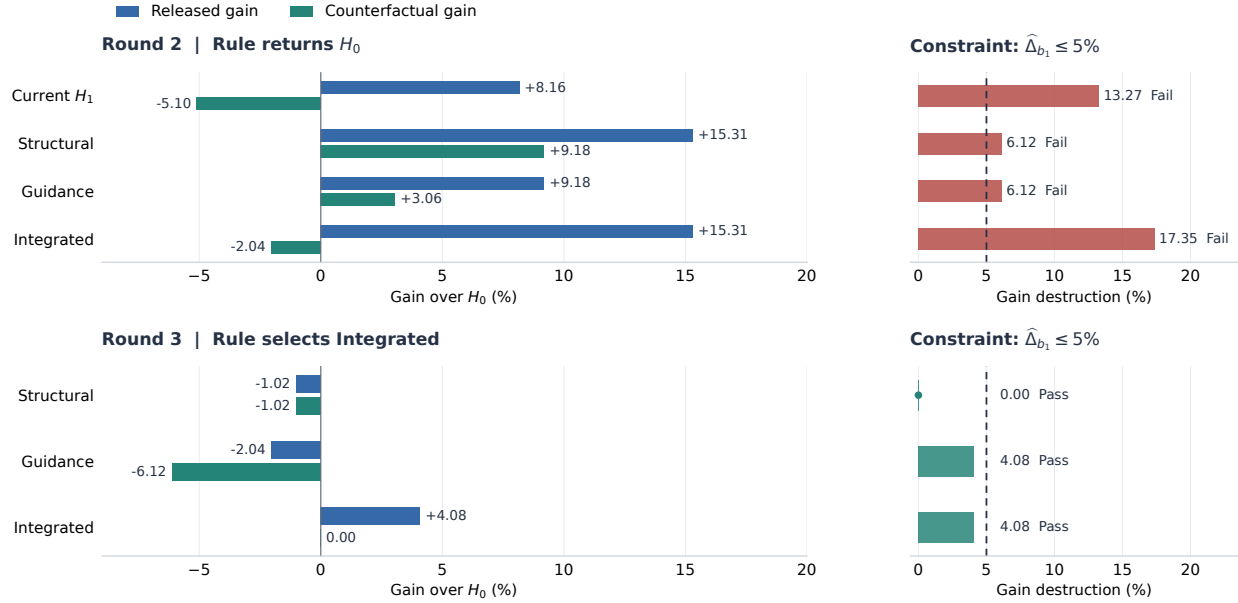


Figure C.2: OfficeQA candidate selection in rounds two and three. Left: released and counterfactual gains over H_0 . Right: gain destruction and the archive constraint $\varepsilon = 0.05$.

Table C.8: Resource use during search. All attempted searches with recorded token use are included, regardless of whether they are retained.

Method	Attempts	Token use (millions)		Wall time
		Total	Excluding cached input	Sum (hours)
HarnessCompass	1,390	283.02	58.59	161.69
CHASE	3,439	688.96	154.42	663.65

D.1 Task Construction, Agent Interface, and Scoring

A task consists of a question, twelve ledger documents, and an arithmetic program that determines the answer. Each ledger record specifies an entity, period, accounting category, approval status, version, accounting basis, and amount. The question identifies the requested entity and relevant periods and restricts the calculation to approved records on the enacted basis. Two evidence documents supply the required amounts. The ten distractors comprise two documents for periods not required by the task, two for an incorrect entity, two with superseded records, two with cancelled or draft records, one with an incorrect accounting basis, and one with an irrelevant category. Each document has six records, including contextual rows outside the requested entity or period.

Table D.1 lists the five task families and shows how their required amounts are distributed across the two evidence documents. Each program contributes 64 tasks and uses the same set of 64 distinct two-digit positive answers. The generator first fixes the answer and then samples amounts satisfying the corresponding arithmetic relation. Combined-total tasks use signed adjustments, and integer-share tasks satisfy $100x/y \in \mathbb{Z}$. Distractor amounts are sampled without using the required amounts. A task is rejected if its answer appears as a complete integer token anywhere in the visible task inputs. The correctness of Syn-Ledger is verified through a separate validation procedure described in Section D.2.

The agent A retrieves evidence through `list_files`, `search`, and `open_file`, and can evaluate arithmetic with `calculator`. Each task allows at most twelve tool calls: four search/list calls in total, six file-opening calls, and two calculator calls. A file-opening call returns three records; another call reads the next three. Search returns four results by default and permits up to twelve.

Table D.1: Syn-Ledger task families. The symbols x, y, z denote the amounts required by each calculation. All answers are integers.

Task family	Calculation	First evidence document	Second evidence document
Period difference	$x - y$	Current appropriation x	Previous appropriation y
Combined total	$x + y$	Capital adjustment x	Operations adjustment y
Net balance	$x + y - z$	Appropriation x , adjustment y	Obligation z
Integer share	$100x/y$	Allocated amount x	Reference total y
Reconciliation gap	$z - (x + y)$	Reported total z	Component amounts x, y

Scoring uses normalized integer exact match. The scorer normalizes surrounding whitespace, signs, and valid comma grouping, then compares the submitted integer with the gold answer. Explanations, multiple numbers, and decimal outputs receive zero. All benchmark variants B_b use the same tools, budgets, and scorer.

Each document is generated from a designated candidate-record block and additional contextual rows, for a total of six records. In an evidence document, the candidate block contains the one or two records required by the arithmetic program. In a distractor, the corresponding block contains structurally matched decoy records that are excluded by the entity, period, approval-status, version, accounting-basis, or category conditions in the question. The remaining records provide context but do not satisfy the requested conditions.

D.2 Benchmark-wide Shortcut Construction and Validation

To construct benchmark-wide shortcuts without changing task semantics, we vary five observable features of each document: filename, directory depth, search rank, candidate-record position, and serialization format (Table D.2). For each feature, we define a favorable and an unfavorable level. Under the released benchmark, required evidence is more often assigned favorable levels, creating associations that a harness can exploit without identifying evidence from document contents. For example, evidence documents may tend to have shorter filenames, shallower directories, or earlier search ranks. These assignments are stored separately from the ledger contents in a rendering manifest and applied by a deterministic renderer. Changing the manifest therefore changes only these observable features while preserving the question, gold answer, and underlying ledger records.

Table D.2: Observable document features used to construct benchmark-wide shortcuts in Syn-Ledger. Search-rank manipulations change only the ordering of broad-query results and do not change document accessibility.

Feature	Favorable level	Unfavorable level
Filename	Short, regular, lexically early	Longer, lexically late
Directory depth	Shallow path	Deeper path
Search rank	Earlier in broad-query results	Later in broad-query results
Candidate-record position	Designated candidate-record block in the first <code>open_file</code> window	Designated candidate-record block in the continuation
Serialization	Fixed-order Markdown table	Equivalent CSV or key-value records

Under B_0 , the five observable features are balanced with respect to evidence membership. In each task, one evidence document and five distractors share one feature configuration, while the other evidence document and five distractors take the opposite level of all five features. Across each 32-task block, the feature configuration assigned to one evidence document cycles through all 2^5 possible combinations. Consequently, every feature configuration occurs twice among the 64 evidence documents and ten times among the 320 distractors. Thus neither any individual feature nor any interaction among the five features is associated with evidence membership under B_0 .

To construct B_{rel} , we modify the feature levels assigned to the two evidence documents while keeping all distractor assignments fixed. For each observable feature, both evidence documents take the favorable level in 24 of the 32 tasks, while the remaining 8 tasks retain one favorable and one unfavorable evidence document. Hence, $56/64 = 7/8$ of the evidence documents have the favorable level for each feature. The eight tasks with one favorable and one unfavorable evidence document are chosen differently for the five features, preventing their favorable assignments from always occurring on the same tasks.

For each observable feature $j \in \{1, \dots, 5\}$, we define a canonical neutralization Φ_j that replaces its assignment under B_{rel} with the corresponding assignment under B_0 , while leaving the other four features unchanged. Each Φ_j is deterministic and idempotent, the five neutralizations mutually commute, and applying all five recovers:

$$B_0 = (\Phi_5 \circ \dots \circ \Phi_1)(B_{\text{rel}}).$$

We additionally define five placebo transformations that alter irrelevant presentation details for construction audits: line endings, metadata order, trailing whitespace, low-rank distractor order, and section labels. The distractor-ordering placebo leaves the rank of every evidence document unchanged. These ten reference transformations are used only to audit the benchmark construction; they are not revealed to the Challenger, which proposes transformations dynamically.

Validation of Syn-Ledger. We validate Syn-Ledger independently of the optimization and evaluation runs. For every task, the gold answer is recomputed from both the canonical records and the rendered documents under B_{rel} , B_0 , the five single-feature neutralizations, and the five placebos, and the two computations must agree. We also verify that both designated evidence documents are necessary: replacing either one while keeping the question, the remaining documents, and all observable-feature assignments fixed must change the resulting legal answer. Additional checks verify the intended feature balance under B_{rel} and B_0 , the composition of the five canonical neutralizations into B_0 , deterministic rendering and replay, and scorer behavior on valid and malformed outputs.

D.3 Preprocessing and Task Allocation

The preceding construction gives each task three distinct attributes: its *task family* specifies what is computed, its documents carry *observable features* that create benchmark-wide shortcuts, and its *task set* specifies when the task is used in the experiment. The 32-task blocks are only a construction device for balancing the five observable features.

The 320 tasks are organized into ten complete 32-task blocks, indexed 0–9. We partition them into D_{evo} , D_{disc} , three round-specific confirmation sets $D_{\text{conf},t}$, and D_{cert} , with sizes $32 + 32 + 3 \times 16 + 208 = 320$ (Table D.3). Without using model scores or trajectories, a deterministic allocator selects one eight-task panel from each of blocks

Table D.3: Syn-Ledger data allocation. Each task appears in exactly one set.

Task set	Size	Construction
D_{evo}	32	Block 0
D_{disc}	32	Block 1
$D_{\text{conf},t}$	16 per round	16 tasks selected from blocks 2–7
D_{cert}	208	The remaining 24 tasks from each of blocks 2–7, plus blocks 8 and 9

2–7. For $t = 1, 2, 3$, the panels selected from blocks $2t$ and $2t + 1$ form $D_{\text{conf},t}$; the 24 unselected tasks in each of these six blocks enter D_{cert} . Blocks 0 and 1 are assigned intact to D_{evo} and D_{disc} , respectively, while blocks 8 and 9 enter D_{cert} intact.

Each selected panel contains one or two tasks from every task family. For each observable feature, exactly 14 of the panel’s $2 \times 8 = 16$ evidence documents take the favorable level under B_{rel} . Each confirmation set therefore has $28/32 = 7/8$ favorable evidence documents for every feature. Because a complete block has 56 favorable evidence documents out of 64 for every feature, the 24 tasks left after selecting a valid panel have $56 - 14 = 42$ favorable evidence documents out of 48. It follows that D_{cert} has:

$$\frac{6 \times 42 + 2 \times 56}{6 \times 48 + 2 \times 64} = \frac{364}{416} = \frac{7}{8}$$

favorable evidence documents for every feature. Figure D.1 summarizes the task construction and deterministic allocation.

All task sets are required to cover the five task families. Exact equality of task-family counts is neither possible nor required because 32, 16, and 208 are not all divisible by five. Among allocations satisfying the panel-level constraints, we minimize the within-set range of task-family counts and then balance the joint favorable/unfavorable patterns of each pair of observable features as evenly as possible; ties are resolved deterministically using a seeded hash of task identifiers. Before model evaluation, we verify set sizes, disjointness, task-family counts, per-feature favorable rates, pairwise feature balance, and deterministic reconstruction. Under our limited budget, we fix $\eta_{\text{conf},t} = 0.05$ and $\varepsilon = 0.05$ on Syn-Ledger.

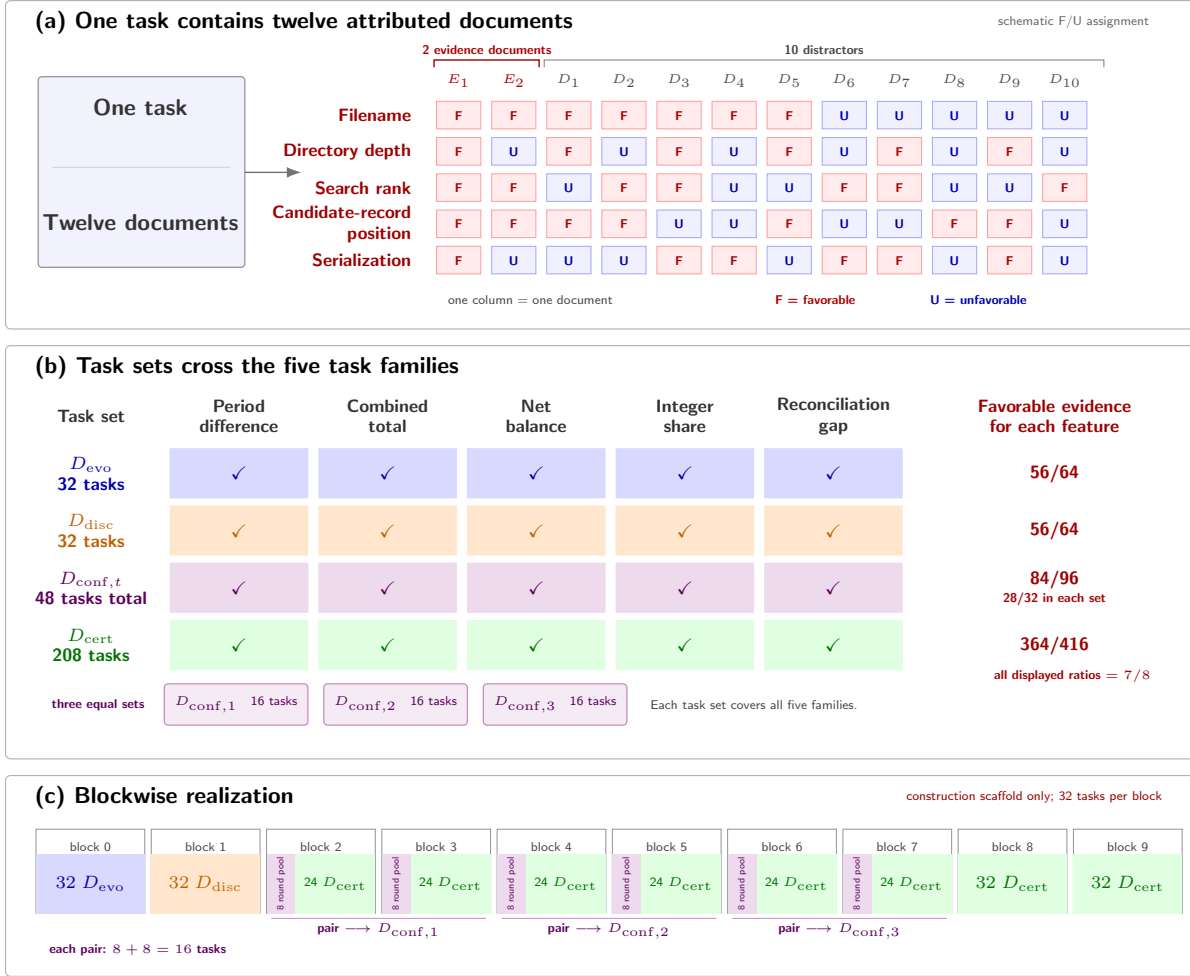


Figure D.1: Syn-Ledger task construction and deterministic allocation.

Supplementary References

Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. EvoSkill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.

Databricks. OfficeQA: A grounded reasoning benchmark suite. <https://github.com/databricks/officeqa>, 2026. Accessed 2026-09-01.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. OfficeQA Pro: An enterprise benchmark for end-to-end grounded reasoning. *arXiv preprint arXiv:2603.08655*, 2026.

Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.442.

Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, Yizhou Jin, and Yan Xu. Harnesscompass: Guiding automatic harness evolution toward generalizable and effective agent harnesses. *arXiv preprint arXiv:2608.01918*, 2026.

Tong Zhang. *Mathematical Analysis of Machine Learning Algorithms*. Cambridge University Press, 2023. doi: 10.1017/9781009093057.